
Các khái niệm cơ bản về đảm bảo chất lượng phần mềm

NỘI DUNG



1. Chất lượng phần mềm
2. Mô hình chữ V
3. Kiểm thử phần mềm

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về **tổng quan** vị trí và vai trò của đảm bảo chất lượng phần mềm trong quá trình phát triển phần mềm
2. Nắm được **các khái niệm** cơ bản trong kiểm thử phần mềm

NỘI DUNG TIẾP THEO



1. Chất lượng phần mềm

2. Mô hình chữ V

3. Kiểm thử phần mềm

1. CHẤT LƯỢNG PHẦN MỀM



- Quan điểm của người dùng: chất lượng là sự **phù hợp với mục đích**
- Quan điểm sản xuất: chất lượng là sự **phù hợp với đặc điểm kỹ thuật**
- Quan điểm sản phẩm: chất lượng gắn liền với **đặc tính vốn có của sản phẩm**
- Quan điểm dựa trên giá trị (kinh tế): chất lượng **phụ thuộc vào số tiền khách hàng sẵn sàng chi trả**

Nói chung, một sản phẩm phần mềm có chất lượng nghĩa là phải đáp ứng được các yêu cầu của nó.

1. CHẤT LƯỢNG PHẦN MỀM



- Chất lượng có nghĩa là một sản phẩm phải đáp ứng được các yêu cầu kỹ thuật của nó.
- Theo định nghĩa của IEEE thì chất lượng phần mềm (IEEE Std 610.12-1990) gồm:
 - Mức độ một hệ thống hay một thành phần hay một tiến trình **đạt được các yêu cầu đặt ra**;
 - Mức độ một hệ thống hay một thành phần hay một tiến trình **đạt được những mong đợi của khách hàng hay người dùng của hệ thống đó**;

NỘI DUNG TIẾP THEO



1. Chất lượng phần mềm

2. Mô hình chữ V

3. Kiểm thử phần mềm

2. MÔ HÌNH CHỮ V



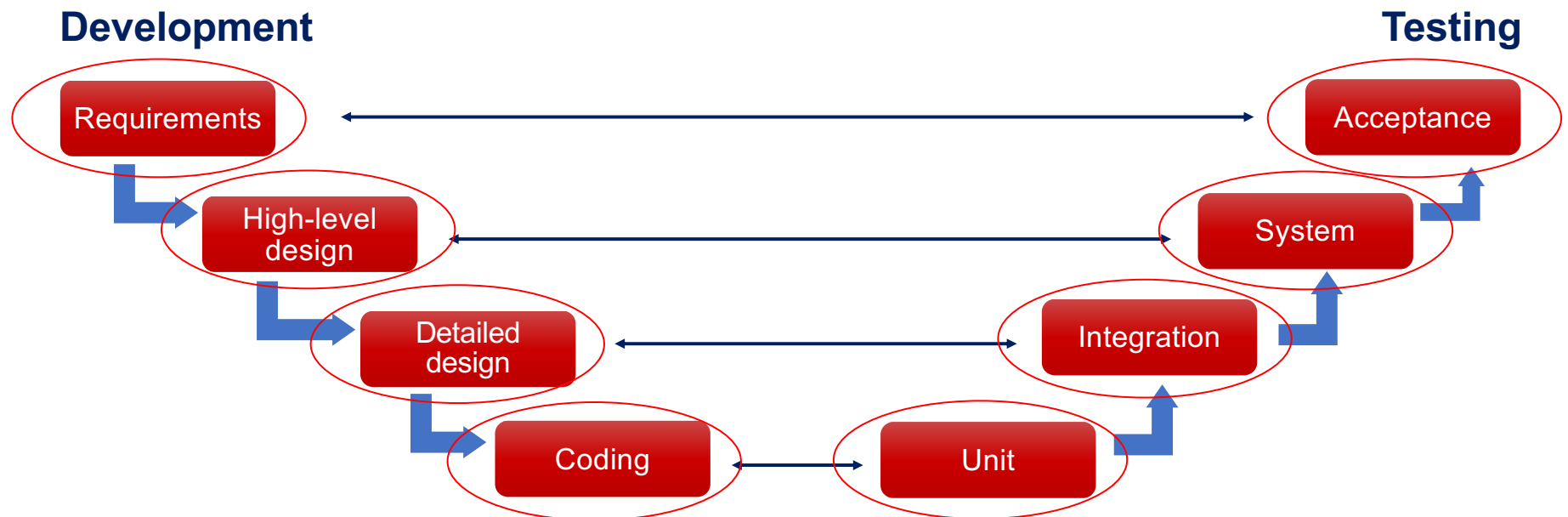
▪ Verification (xác minh)

- Xác minh xem **hệ thống có được xây dựng đúng hay không** (*verify if the system is built right*)
- Là hoạt động đánh giá phần mềm bằng cách xác định xem hệ thống có **thoả mãn các yêu cầu nêu ra ban đầu hay không** (*from perspective of development team*)

▪ Validation (xác thực)

- Kiểm tra xem ta đã **xây dựng đúng hệ thống hay chưa** (*validate if we built a right system*)
- Là hoạt động kiểm tra xem sản phẩm phần mềm có **thoả mãn các yêu cầu của người dùng hay không** (*from perspective of users*)

2. MÔ HÌNH CHỮ V



Hình 2.1: Mô hình chữ V

NỘI DUNG TIẾP THEO



1. Chất lượng phần mềm

2. Mô hình chữ V

3. Kiểm thử phần mềm

3.1. Các thuật ngữ về kiểm thử

3.2. Khó khăn trong kiểm thử phần mềm

3.3. Những lưu ý khi kiểm thử

3.4. Các kỹ thuật kiểm thử

3. KIỂM THỬ PHẦN MỀM



3.1. Các thuật ngữ về kiểm thử

- **Kiểm thử (software testing):** là một quy trình phân tích và đánh giá một thành phần phần mềm (software item) để **phát hiện sự khác biệt giữa thành phần đó với các yêu cầu đặt ra** (gọi là lỗi)

3. KIỂM THỬ PHẦN MỀM



3.1. Các thuật ngữ về kiểm thử

- **Ca kiểm thử (test case):** là một cặp *<input, expected outcome>*
- Khi thực hiện một ca kiểm thử người ta sẽ đưa input tương ứng vào chương trình và kiểm tra kết quả trả về của hệ thống
 - **Passed test:** là trạng thái sau khi thực hiện một ca kiểm thử **thu được kết quả trả về tương ứng với kết quả mong đợi**
 - **Failed test:** **ngược lại**
- **Test suite:** là một **tập hợp các test case được xây dựng để kiểm thử cho 1 chức năng hoặc 1 nhóm chức năng của phần mềm**

3. KIỂM THỬ PHẦN MỀM



3.2. Khó khăn trong kiểm thử phần mềm

- Nâng cao chất lượng phần mềm nhưng **không vượt quá chất lượng khi thiết kế** vì chỉ phát hiện các lỗi tiềm tàng và sửa chúng
- Phát hiện lỗi bị hạn chế do được **thực hiện thử công là chính**
- Dễ bị **ảnh hưởng tâm lý** khi kiểm thử
- **Khó đảm bảo tính đầy đủ** của kiểm thử

3. KIỂM THỬ PHẦN MỀM



3.3. Những lưu ý khi kiểm thử

- Chất lượng phần mềm **do khâu thiết kế quyết định là chủ yếu**, chứ không phải khâu kiểm thử;
- Tính dễ kiểm thử **phụ thuộc vào cấu trúc của chương trình**;
- Người kiểm thử và người phát triển **nên khác nhau**;
- Dữ liệu thử cho kết quả bình thường thì không mang lại nhiều ý nghĩa, **cần có những dữ liệu kiểm thử để phát hiện ra lỗi**;

3. KIỂM THỬ PHẦN MỀM



3.3. Những lưu ý khi kiểm thử

- Khi thiết kế trường hợp thử, không chỉ cần dữ liệu đầu vào, mà phải thiết kế trước cả dữ liệu kết quả sẽ có;
- Khi phát sinh thêm trường hợp thử thì nên thử lại những trường hợp kiểm thử trước đó để tránh ảnh hưởng lan truyền.

3. KIỂM THỬ PHẦN MỀM



3.4. Các kỹ thuật kiểm thử

- Có 2 kỹ thuật kiểm thử cơ bản:
 - Kiểm thử hộp đen – **black box testing**
 - Kiểm thử hộp trắng – **white box testing**
- Cả hai kỹ thuật này đều **yêu cầu phải thực thi chương trình**
- Cần thiết kế các ca kiểm thử để có thể **phát hiện nhiều lỗi nhất**
- Việc sử dụng kỹ thuật nào phụ thuộc vào các giai đoạn (các mức kiểm thử khác nhau) trong quá trình phát triển phần mềm

TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học một số **khái niệm cơ bản** trong đảm bảo chất lượng phần mềm và kiểm thử phần mềm
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về các kỹ thuật kiểm thử

Phương pháp kiểm thử hộp trắng (phần 1)

NỘI DUNG



1. Khái niệm kiểm thử hộp trắng
2. Kiểm thử luồng điều khiển

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về **kỹ thuật kiểm thử hộp trắng**
2. Nắm được **phương pháp kiểm thử luồng điều khiển**

NỘI DUNG TIẾP THEO



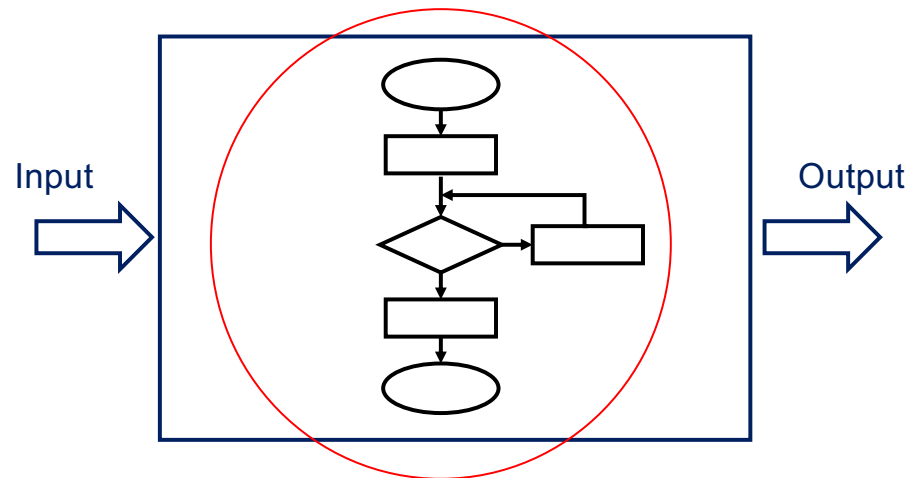
1. Khái niệm kiểm thử hộp trắng

2. Kiểm thử luồng điều khiển

1. KHÁI NIỆM KIỂM THỬ HỘP TRẮNG



- Kiểm thử hộp trắng là chiến lược kiểm thử dựa trên **cấu trúc chức năng** (structural testing) nhằm **kiểm tra về mặt thuật toán và cấu trúc bên trong** của sản phẩm phần mềm.

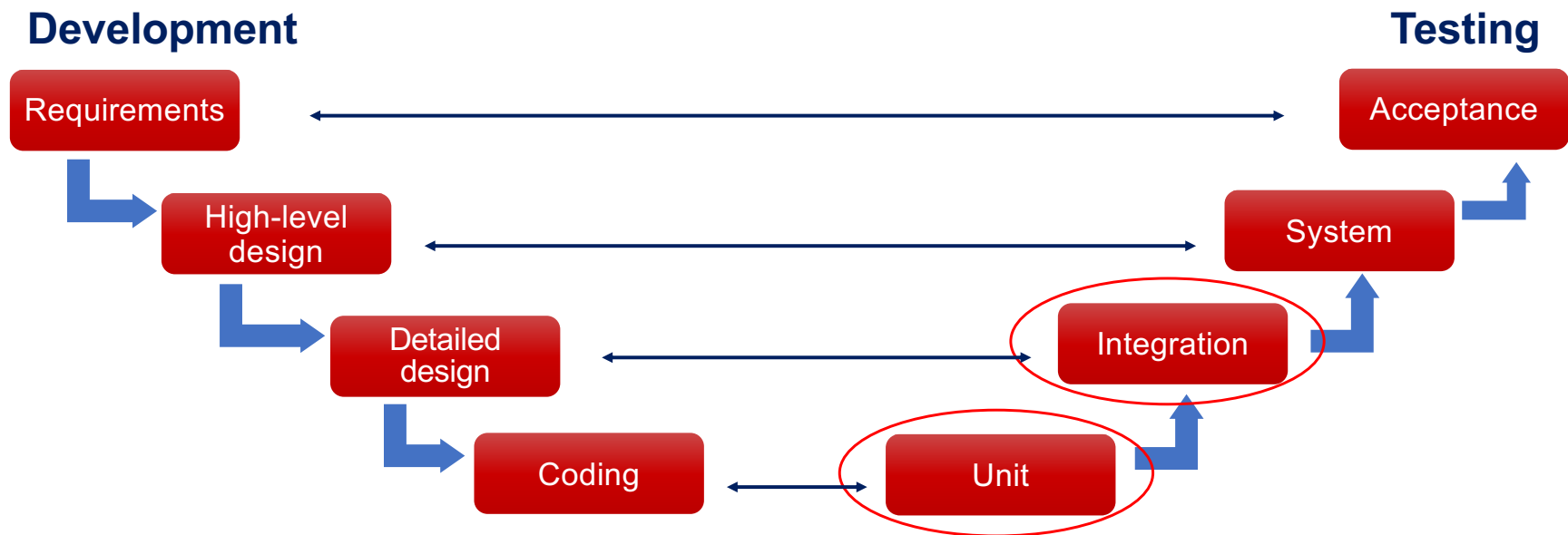


Hình 1.1: Kiểm thử hộp trắng

1. KHÁI NIỆM KIỂM THỬ HỘP TRẮNG



- Được thực hiện bởi lập trình viên và các kiểm thử viên;
- Thực hiện chủ yếu ở giai đoạn kiểm thử đơn vị và kiểm thử tích hợp.



Hình 1.2: Mô hình chữ V và kiểm thử hộp trắng

1. KHÁI NIỆM KIỂM THỬ HỘP TRẮNG



Các phương pháp thiết kế ca kiểm thử

- ***Kiểm thử luồng điều khiển***
 - Bao phủ tất cả đường dẫn
 - Bao phủ lệnh
 - Bao phủ nhánh
- ***Kiểm thử luồng dữ liệu***
 - Bao phủ tất cả các điểm định nghĩa của biến
 - Bao phủ tất cả các điểm sử dụng của biến
 - ...

NỘI DUNG TIẾP THEO



1. Khái niệm kiểm thử hộp trắng

2. Kiểm thử luồng điều khiển

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Đồ thị luồng điều khiển (Control Flow Graph - CFG)

- Đồ thị luồng điều khiển thể hiện tập hợp tất cả các đường dẫn từ điểm bắt đầu tới điểm kết thúc của chương trình được biểu diễn bằng đồ thị;
- Biểu diễn cấu trúc của một đơn vị chương trình.

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Đồ thị luồng điều khiển (CFG)

- Một đường dẫn (path) của chương trình
 - Là một chuỗi các câu lệnh (statement) từ điểm bắt đầu cho đến điểm kết thúc của chương trình
 - Với mỗi một dữ liệu đầu vào (input data), chương trình có thể thực thi theo các đường dẫn khác nhau (execution path)
- Mục đích của việc thiết kế các ca kiểm thử
 - Tìm ra những đường dẫn (path) có lỗi hoặc tiềm ẩn nhiều lỗi nhất có thể

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Các tiêu chí lựa chọn đường dẫn chương trình

- *Bao phủ toàn bộ các đường dẫn (vét cạn)*
 - Nếu chương trình có 10 lệnh rẽ nhánh thì số đường dẫn tối đa là $2^{10} = 1024$ đường dẫn, nên có 1024 test case.
- *Bao phủ lệnh*
 - Đảm bảo mọi câu lệnh trong chương trình đều được kiểm tra
- *Bao phủ nhánh*
 - Đảm bảo mọi nhánh rẽ tương ứng với giá trị true/false ở câu lệnh điều kiện trong chương trình đều được kiểm thử.

2. KIỂM THỬ LƯỖNG ĐIỀU KHIỂN



Ví dụ

- Cho đoạn chương trình sau

```
bool foo(int a, int b) {  
    bool c;  
    if (a==0)  
        c = b < 5;  
    else  
        c = b < 0;  
    return c;  
}
```

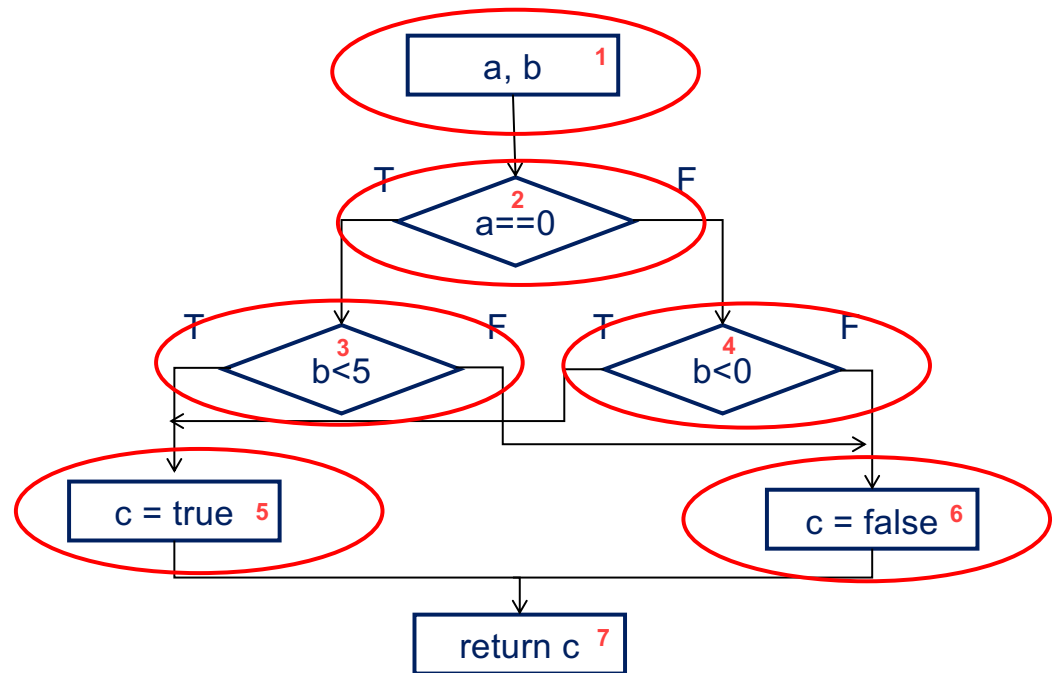
2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Ví dụ

- Vẽ sơ đồ luồng điều khiển

```
bool foo(int a, int b) {  
    bool c;  
    if (a==0)  
        c = b < 5;  
    else  
        c = b < 0;  
    return c;  
}
```



Hình 2.1: Sơ đồ luồng điều khiển

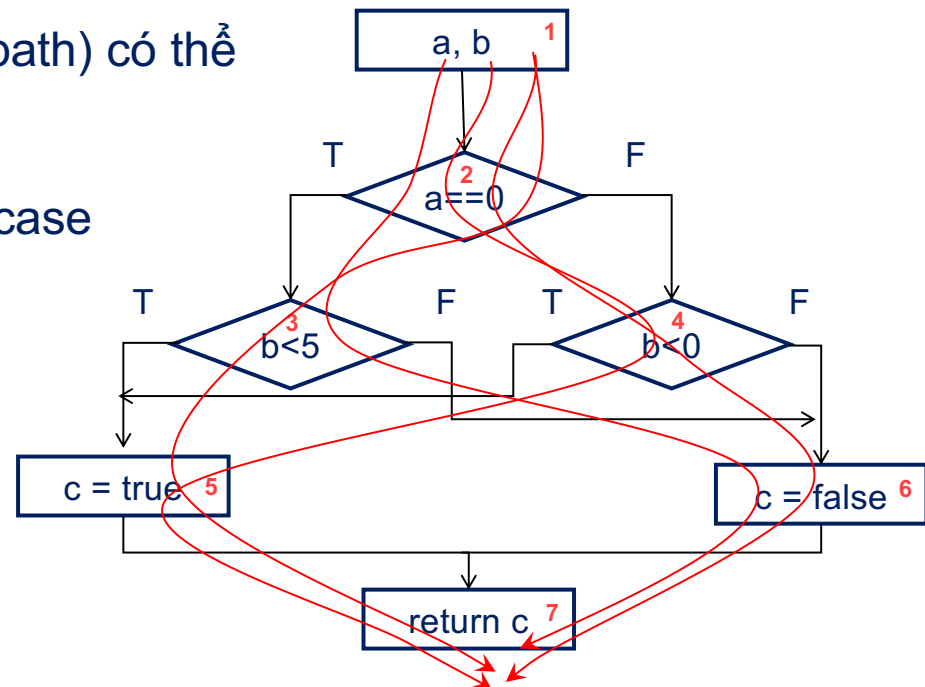
2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Tiêu chí phủ đường dẫn

- Lựa chọn tất cả các đường dẫn (path) có thể có trong chương trình
- Mỗi path sẽ tương ứng với 1 test case
- Xác định các đường dẫn

- 1) 1-2-3-5-7
- 2) 1-2-3-6-7
- 3) 1-2-4-5-7
- 4) 1-2-4-6-7



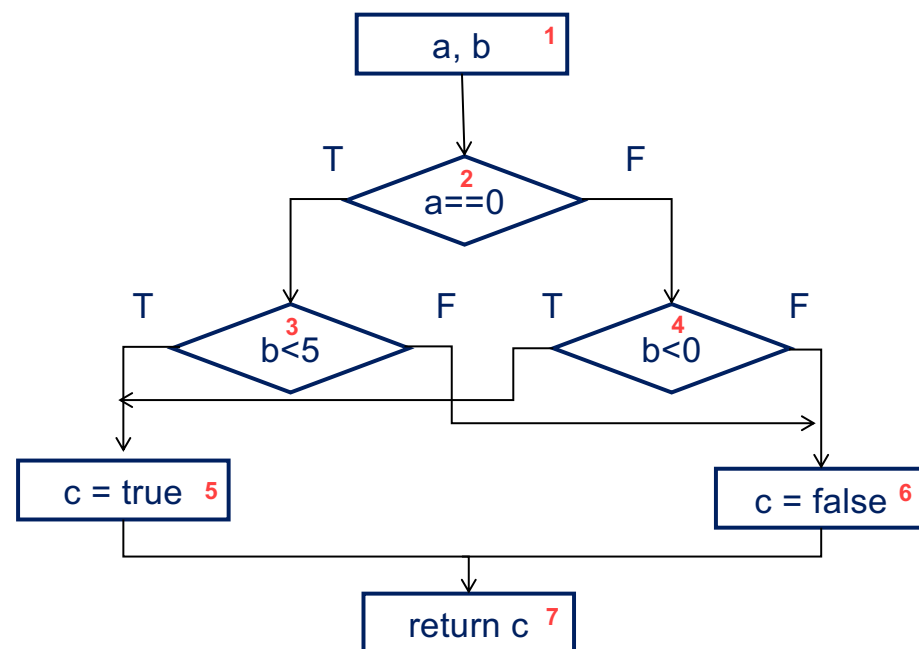
Hình 2.1: Sơ đồ luồng điều khiển

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Tiêu chí phủ đường dẫn

	a==0	b<5	b<0	return
1	T	T	T	T
2	T	T	F	T
<u>3</u>	<u>I</u>	<u>F</u>	<u>I</u>	-
4	T	F	F	F
5	F	T	T	T
<u>6</u>	<u>F</u>	<u>F</u>	<u>I</u>	-
7	F	T	F	F
8	F	F	F	F



Hình 2.1: Sơ đồ luồng điều khiển

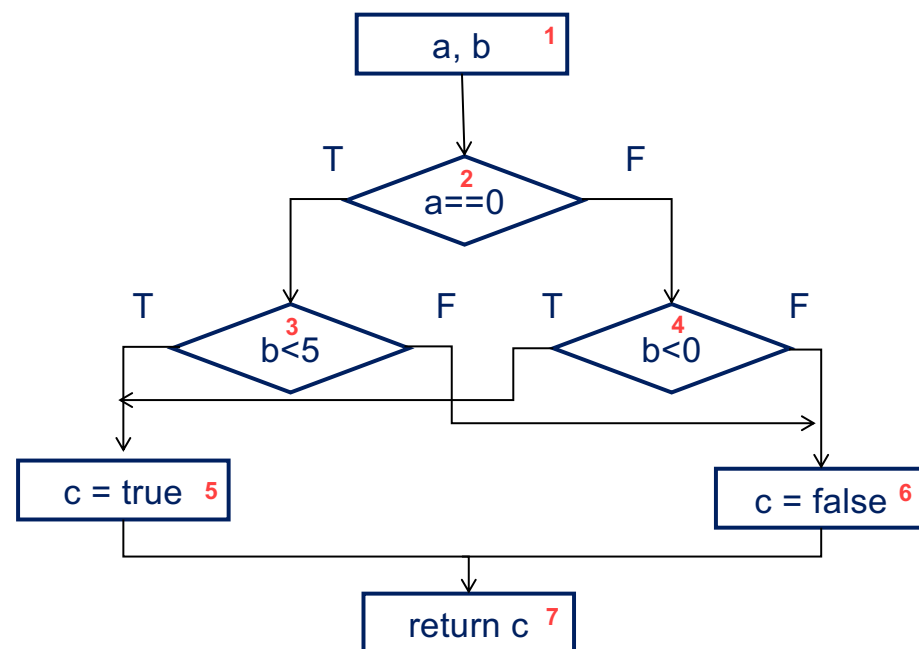
2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Tiêu chí phủ đường dẫn

	a==0	b<5	b<0	return
1)	1	T	T	T
	2	T	F	T
	3	F	F	-
2)	4	T	F	F
3)	5	F	T	T
	6	F	F	-
4)	7	F	T	F
	8	F	F	F

- 1) **<T,T,*>** 1-2(T)-3(T)-5-7
 2) **<T,F,F>** 1-2(T)-3(F)-6-7
 3) **<F,T,T>** 1-2(F)-4(T)-5-7
 4) **<F*,F>** 1-2(F)-4(F)-6-7



Hình 2.1: Sơ đồ luồng điều khiển

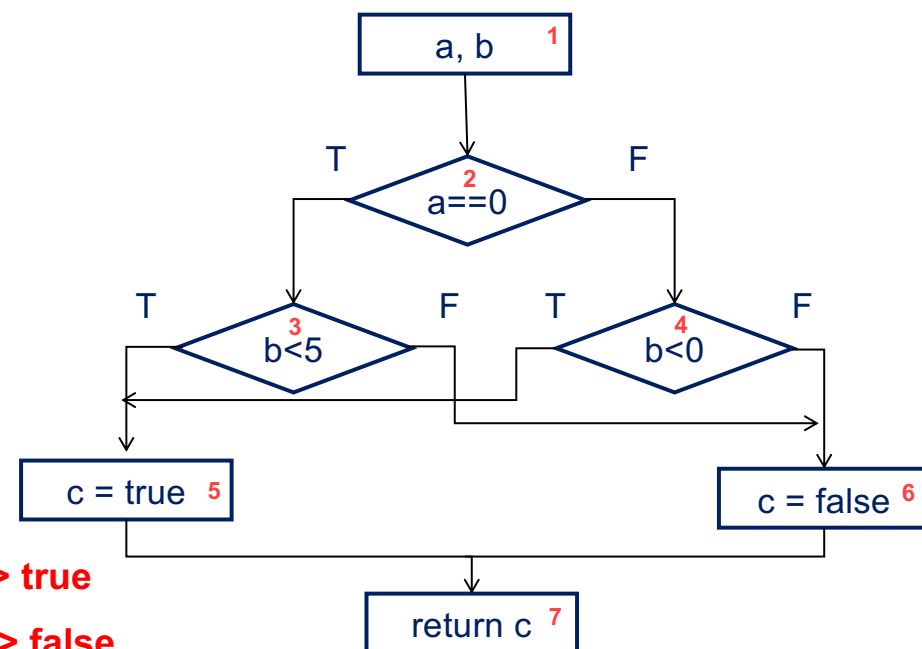
2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Tiêu chí phủ đường dẫn

	a==0	b<5	b<0	return
1)	1	T	T	T
	2	T	F	T
	3	F	F	-
2)	4	T	F	F
3)	5	F	T	T
	6	F	F	-
4)	7	F	T	F
	8	F	F	F

- 1) $\langle T, T, * \rangle$ 1-2(T)-3(T)-5-7 $\Rightarrow$ TC: $\text{foo}(0, 3) \Rightarrow \text{true}$
- 2) $\langle T, F, F \rangle$ 1-2(T)-3(F)-6-7 $\Rightarrow$ TC: $\text{foo}(0, 6) \Rightarrow \text{false}$
- 3) $\langle F, T, T \rangle$ 1-2(F)-4(T)-5-7 $\Rightarrow$ TC: $\text{foo}(1, -2) \Rightarrow \text{true}$
- 4) $\langle F, *, F \rangle$ 1-2(F)-4(F)-6-7 $\Rightarrow$ TC: $\text{foo}(1, 3) \Rightarrow \text{false}$



Hình 2.1: Sơ đồ luồng điều khiển

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Nhận xét (1)

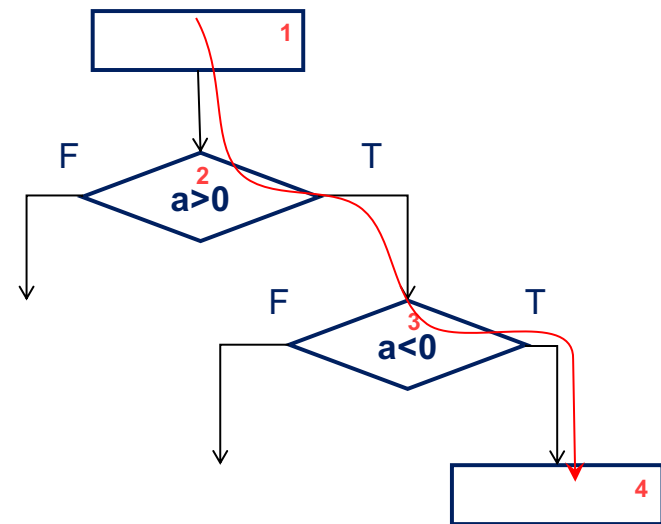
- Với bài toán kiểm thử đường dẫn ta cần xây dựng đồ thị luồng điều khiển
- Cần xác định các đường dẫn và các giá trị cần thiết của biến/tham số để thiết kế các test case

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Nhận xét (2)

- Có thể tồn tại một số đường dẫn không thể thiết kế được test case ví dụ không thể thực thi đường dẫn đi qua các node 1, 2, 3, 4 như ở hình bên



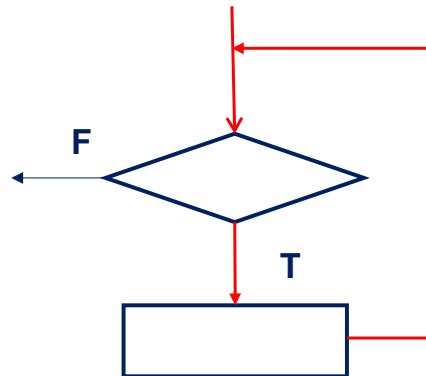
Hình 2.2: Đường đi không hợp lệ

2. KIỂM THỬ LUỒNG ĐIỀU KHIỂN



Nhận xét (3)

- Trong trường hợp chương trình có câu lệnh lặp (đặc biệt với câu lệnh lặp không xác định) thì số lượng test case sẽ rất lớn (hoặc không thể xác định được).



TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học **kĩ thuật kiểm thử hộp trắng, phương pháp kiểm thử luồng điều khiển** (tiêu chí phủ đường dẫn)
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về tiêu chí bao phủ lệnh và bao phủ nhánh

Phương pháp kiểm thử hộp trắng (phần 2)

NỘI DUNG



1. Kỹ thuật bao phủ lệnh
2. Kỹ thuật bao phủ nhánh
3. So sánh các tiêu chí

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về **kỹ thuật bao phủ lệnh**
2. Nắm được **phương pháp kiểm thử bao phủ nhánh**

NỘI DUNG TIẾP THEO



1. Kỹ thuật bao phủ lệnh

2. Kỹ thuật bao phủ nhánh

3. So sánh các tiêu chí

1. KỸ THUẬT BAO PHỦ LỆNH



Mục tiêu

- Đảm bảo **mỗi câu lệnh** đều được thực thi ít nhất 1 lần
- Như vậy: mọi đỉnh của đồ thị luồng điều khiển đều được **xuất hiện ít nhất 1 lần** trong các đường dẫn

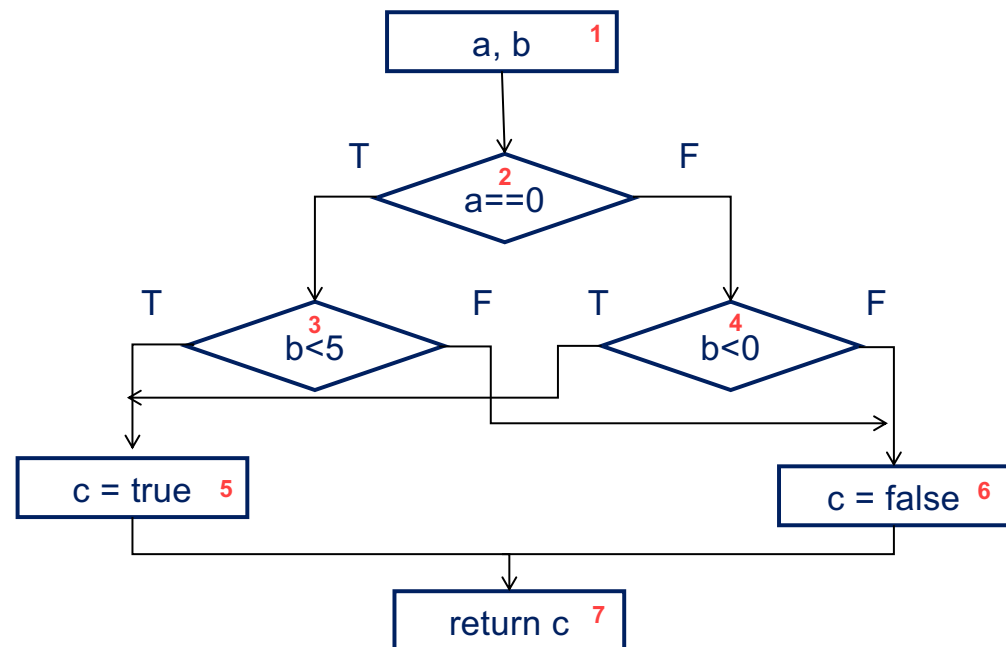
1. KỸ THUẬT BAO PHỦ LỆNH



Ví dụ

- Cho đoạn chương trình

```
bool foo(int a, int b) {  
    bool c;  
    if (a==0)  
        c = b < 5;  
    else  
        c = b < 0;  
    return c;  
}
```



Hình 1.1: Sơ đồ luồng điều khiển

1. KỸ THUẬT BAO PHỦ LỆNH



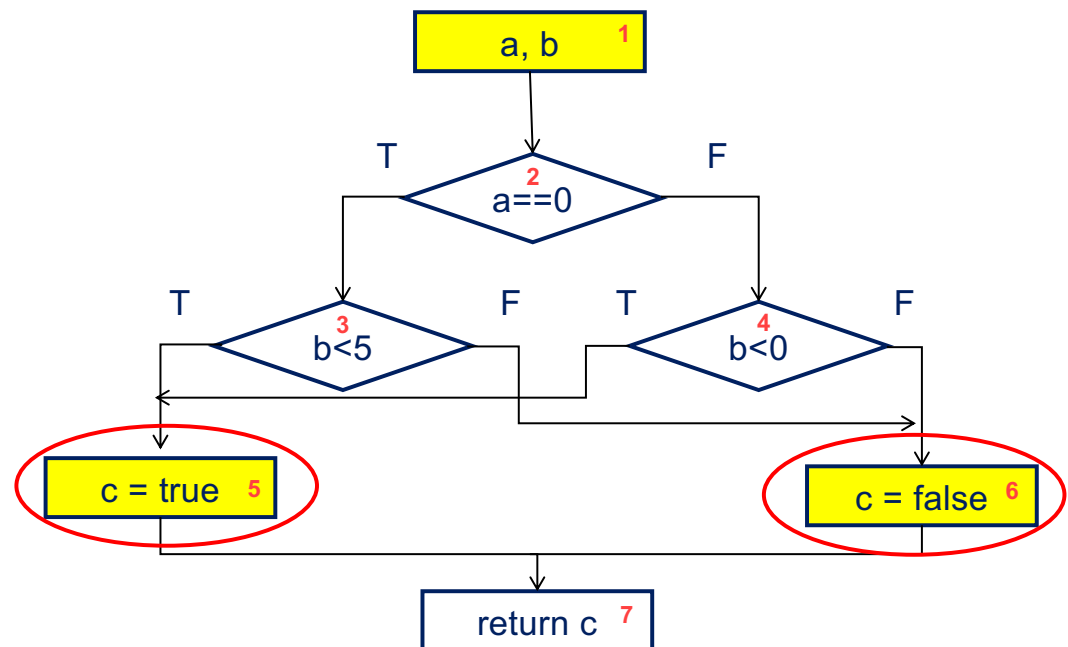
Ví dụ

- Cho đoạn chương trình

```
bool foo(int a, int b) {  
    bool c;  
    if (a==0)  
        c = b < 5;  
    else  
        c = b < 0;  
    return c;  
}
```

1. TC $\text{foo}(0,3) \Rightarrow \text{true}$

2. TC $\text{foo}(1,3) \Rightarrow \text{false}$



Hình 1.1: Sơ đồ luồng điều khiển

NỘI DUNG TIẾP THEO



1. Kỹ thuật bao phủ lệnh

2. Kỹ thuật bao phủ nhánh

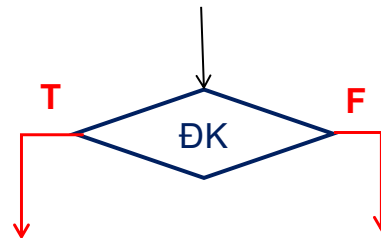
3. So sánh các tiêu chí

2. KỸ THUẬT BAO PHỦ NHÁNH



Mục tiêu

- Đảm bảo **mỗi điểm quyết định** trên đồ thị (điểm tại đó có nhánh rẽ) đều được kiểm thử
 - ít nhất 1 lần với nhánh tương ứng với giá trị điều kiện là *true* và
 - ít nhất 1 lần với nhánh có giá trị điều kiện là *false*.



2. KỸ THUẬT BAO PHỦ NHÁNH

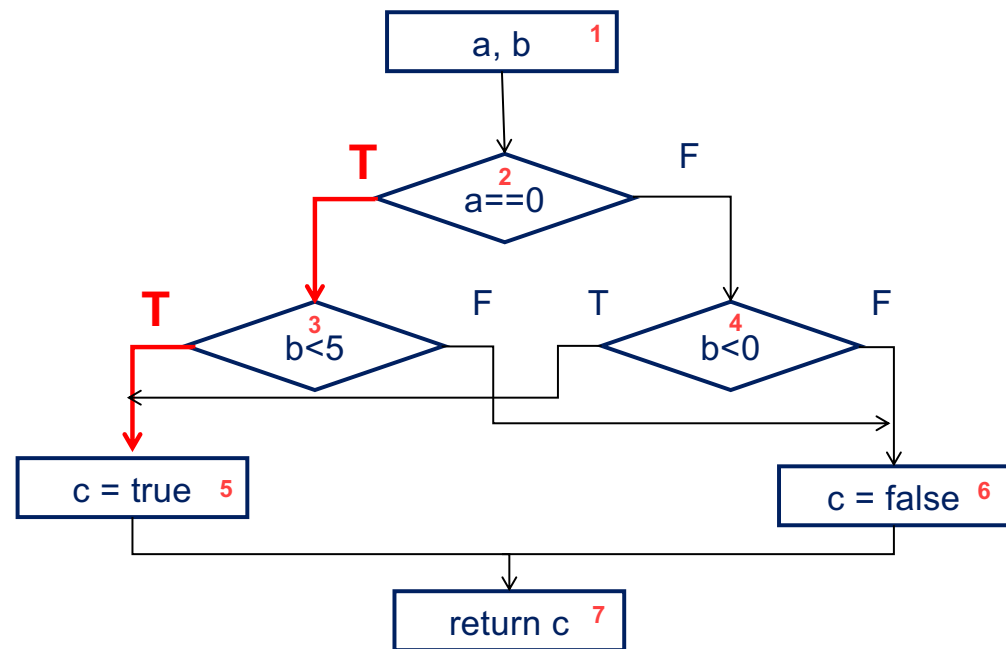


Ví dụ

- Trường hợp 1

```
bool foo(int a, int b) {  
    bool c;  
    if (a==0)  
        c = b < 5;  
    else  
        c = b < 0;  
    return c;  
}
```

TC: foo(0,3) => true



Hình 2.1: $a==0$ (T) và $b<5$ (T)

2. KỸ THUẬT BAO PHỦ NHÁNH

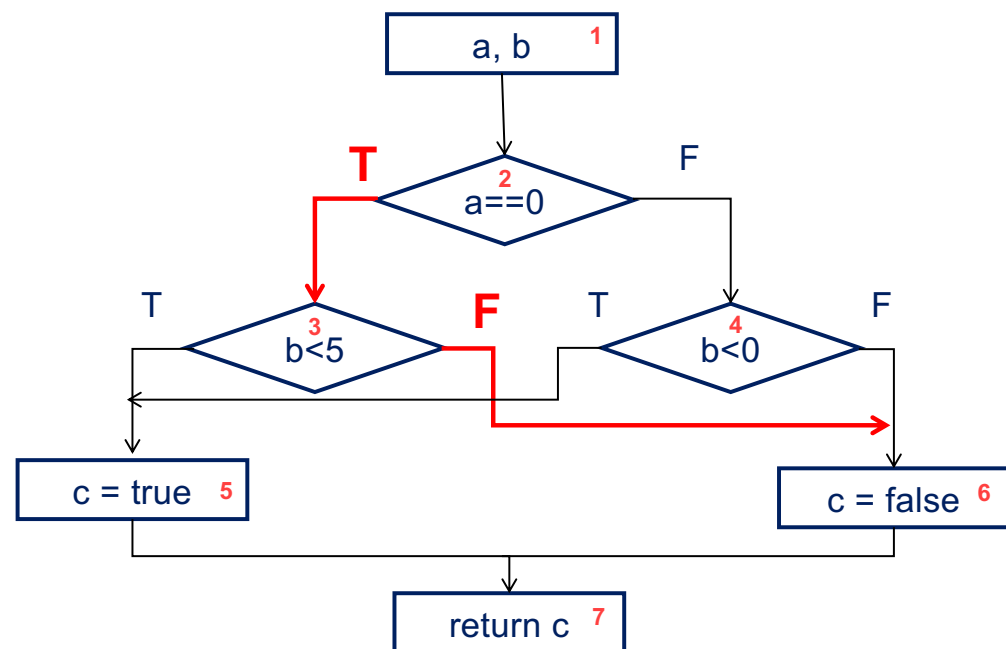


Ví dụ

- Trường hợp 2

```
bool foo(int a, int b) {  
    bool c;  
    if (aT==0)  
        c = b F< 5;  
    else  
        c = b < 0;  
    return c;  
}
```

TC: foo(0,6) => false



Hình 2.2: `a==0` (T) và `b<5` (F)

2. KỸ THUẬT BAO PHỦ NHÁNH

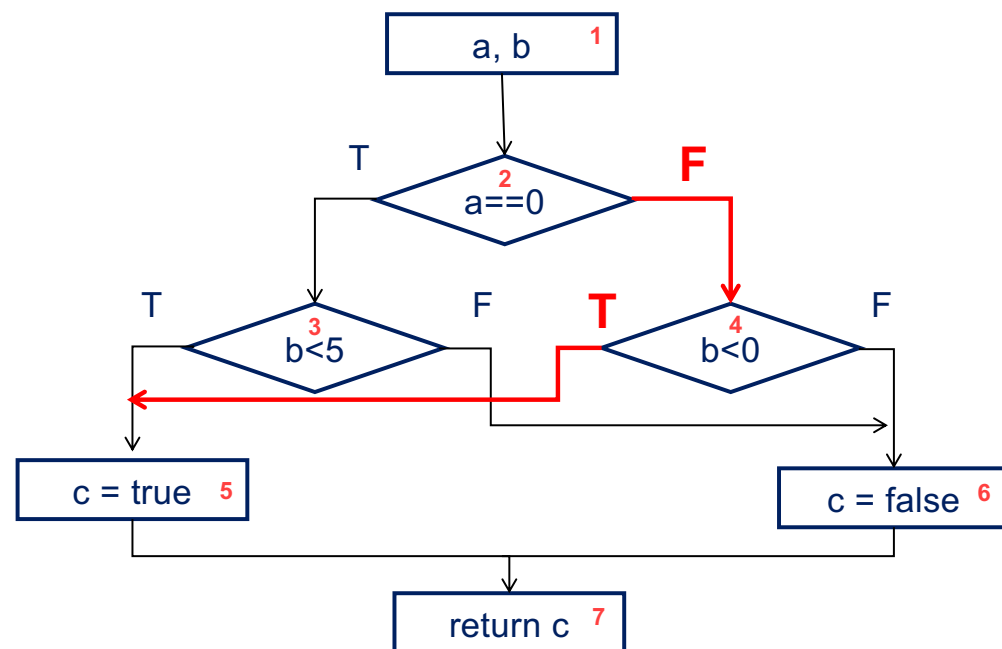


Ví dụ

- Trường hợp 3

```
bool foo(int a, int b) {  
    bool c;  
    if (aF==0)  
        c = b < 5;  
    else  
        c = b T< 0;  
    return c;  
}
```

TC: foo(1,-2) => true



Hình 2.3: `a==0` (F) và `b<0` (T)

2. KỸ THUẬT BAO PHỦ NHÁNH

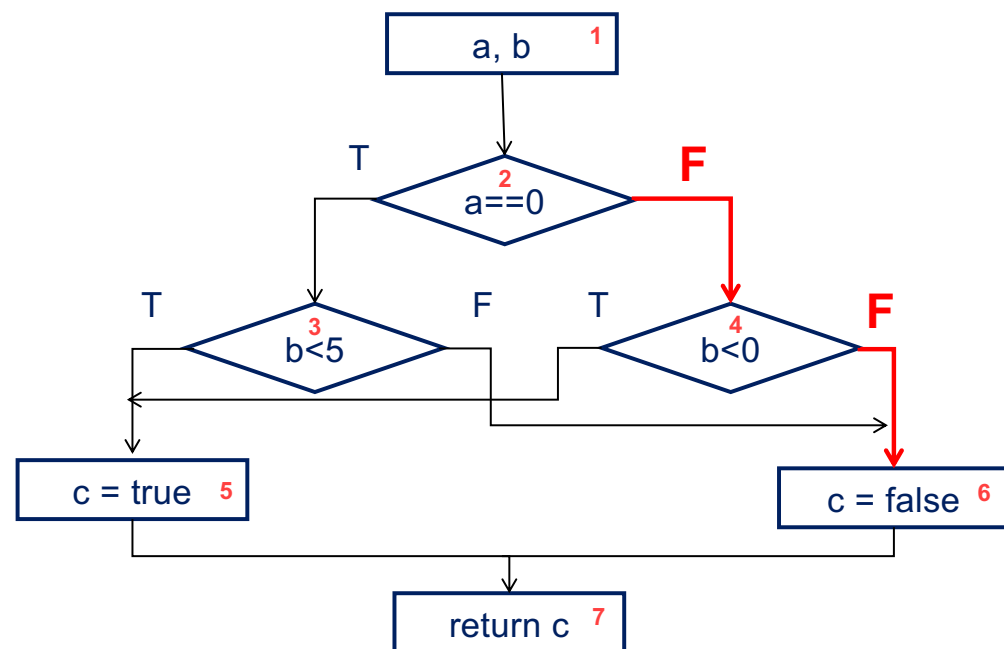


Ví dụ

- Trường hợp 4

```
bool foo(int a, int b) {  
    bool c;  
    if (aF==0)  
        c = b < 5;  
    else  
        c = b F< 0;  
    return c;  
}
```

TC: $\text{foo}(1,3) \Rightarrow \text{false}$



Hình 2.4: $a==0$ (F) và $b<0$ (F)

NỘI DUNG TIẾP THEO



1. Kỹ thuật bao phủ lệnh
2. Kỹ thuật bao phủ nhánh

3. So sánh các tiêu chí

3. SO SÁNH CÁC TIÊU CHÍ



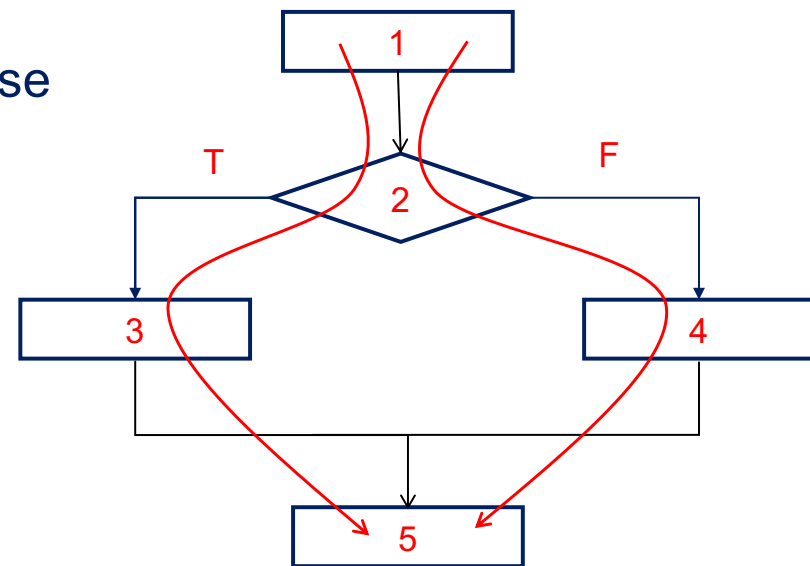
- Tiêu chí mạnh nhất: bao phủ mọi đường dẫn
- Tiêu chí yếu nhất: bao phủ mọi câu lệnh
 - Với các câu lệnh rẽ nhánh có thể chỉ cần kiểm thử 1 nhánh tương ứng với giá trị true/false
 - Thường được kết hợp thêm tiêu chí bao phủ điểm quyết định.

3. SO SÁNH CÁC TIÊU CHÍ



Ví dụ 1

- Các tiêu chí
 - Bao phủ đường dẫn: 2 test case
 - Bao phủ nhánh: 2 test case
 - Bao phủ lệnh: 2 test case
- Các test case
 - TC1: 1-2-3-5
 - TC2: 1-2-4-5



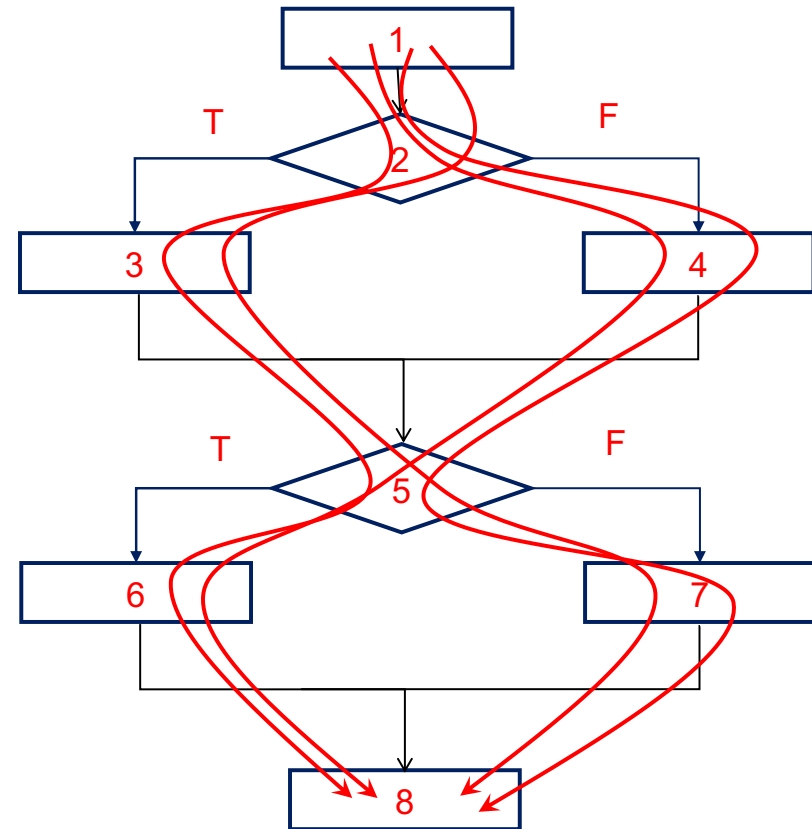
Hình 3.1: Sơ đồ luồng điều khiển ví dụ 1

3. SO SÁNH CÁC TIÊU CHÍ



Ví dụ 2

- Bao phủ đường dẫn: 4 test case
- Các test case
 - TC1: 1-2-3-5-6-8
 - TC2: 1-2-3-5-7-8
 - TC3: 1-2-4-5-6-8
 - TC4: 1-2-4-5-7-8



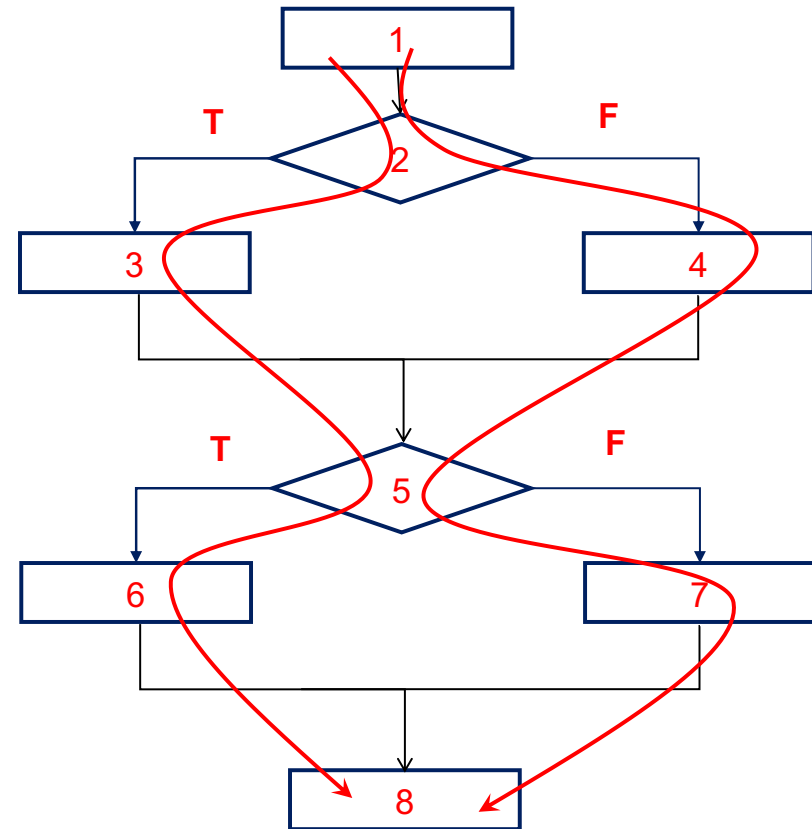
Hình 3.2: Sơ đồ luồng điều khiển ví dụ 2

3. SO SÁNH CÁC TIÊU CHÍ



Ví dụ 2

- Bao phủ nhánh: 2 test case
- Các test case
 - TC1: 1-2-3-5-6-8
 - TC2: 1-2-4-5-7-8



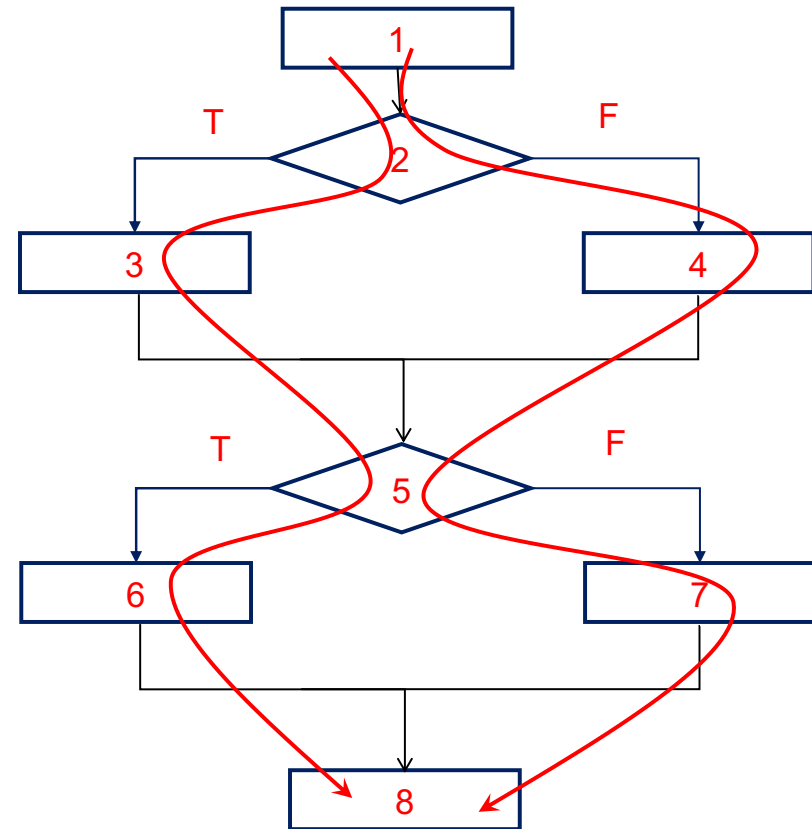
Hình 3.2: Sơ đồ luồng điều khiển ví dụ 2

3. SO SÁNH CÁC TIÊU CHÍ



Ví dụ 2

- Bao phủ lệnh: 2 test case
- Các test case
 - TC1: 1-2-3-5-6-8
 - TC2: 1-2-4-5-7-8



Hình 3.2: Sơ đồ luồng điều khiển ví dụ 2

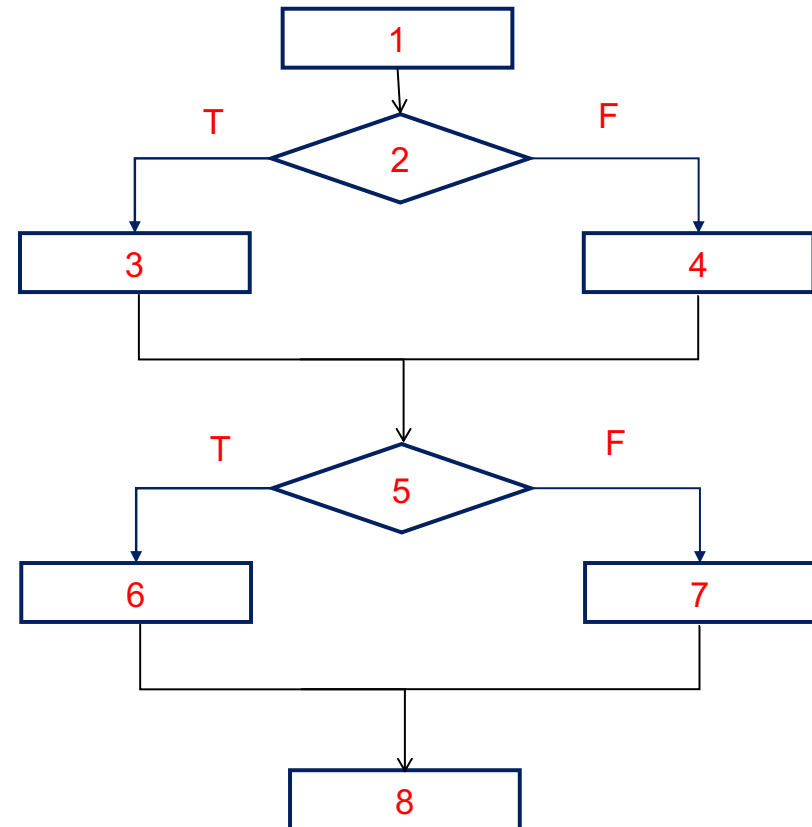
3. SO SÁNH CÁC TIÊU CHÍ



Nhận xét

- Bao phủ đường dẫn: 4 test case
- Bao phủ nhánh: 2 test case
- Bao phủ lệnh: 2 test case

Phủ đường dẫn: mạnh nhất



TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học một số **kỹ thuật bao phủ lệnh và bao phủ nhánh** trong kiểm thử hộp trắng
2. Sau bài này, **người học có thể tự tìm hiểu thêm** về kỹ thuật kiểm thử theo luồng dữ liệu

Bài tập: Phương pháp kiểm thử hộp trắng

NỘI DUNG



1. Bài toán kiểm thử với đồ thị luồng điều khiển
2. Bài tập áp dụng

MỤC TIÊU



Sau bài học này, người học có thể:

1. Nắm được **phương pháp** giải quyết bài toán kiểm thử với đồ thị luồng điều khiển
2. Biết **vận dụng kiến thức** về kỹ thuật kiểm thử cho bài toán kiểm thử với các tiêu chí bao phủ đường dẫn, bao phủ nhánh và bao phủ câu lệnh

NỘI DUNG TIẾP THEO



1. Bài toán kiểm thử với đồ thị luồng điều khiển

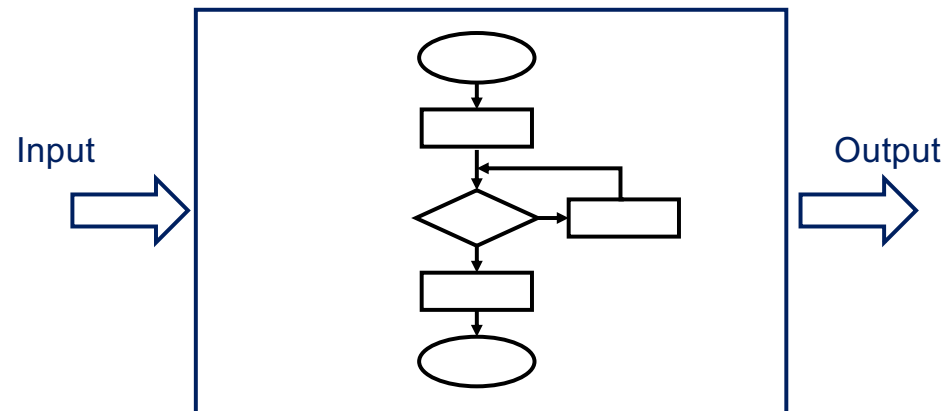
2. Bài tập áp dụng

1. BÀI TOÁN KIỂM THỬ HỘP TRẮNG



Bài toán kiểm thử luồng điều khiển

- Mô tả
 - Cho một đoạn chương trình (mã nguồn)
 - Thiết kế các ca kiểm thử đảm bảo bao phủ đường dẫn (/nhánh/câu lệnh)



Hình 1.1: Kiểm thử hộp trắng

1. BÀI TOÁN KIỂM THỬ HỘP TRẮNG



Bài toán kiểm thử luồng điều khiển

▪ Cách giải quyết bài toán

- Bước 1: Vẽ đồ thị CFG tương ứng với đoạn chương trình đã cho.
- Bước 2: Xác định các đường đi (/nhánh rẽ/câu lệnh) tương ứng theo yêu cầu của đề bài.
- Bước 3: Xác định các giá trị đầu vào tương ứng theo các đường đi (/nhánh rẽ/câu lệnh) ở bước 2.
- Bước 4: Xác định các test case tương ứng với các giá trị tìm được ở bước 3.

2. BÀI TẬP ÁP DỤNG



Ví dụ 1: giảm giá sản phẩm

Cho chương trình để xét giảm giá sản phẩm với mô tả như sau:

- Đầu vào:
 - có là thành viên hay không
 - có mã khuyến mãi hay không
- Đầu ra:
 - giảm 10% khi là thành viên hoặc giảm 7% nếu chưa phải là thành viên
 - giảm thêm 5% nếu có mã khuyến mãi hoặc chỉ giảm 3% nếu không có mã

2. BÀI TẬP ÁP DỤNG



Đoạn mã chương trình

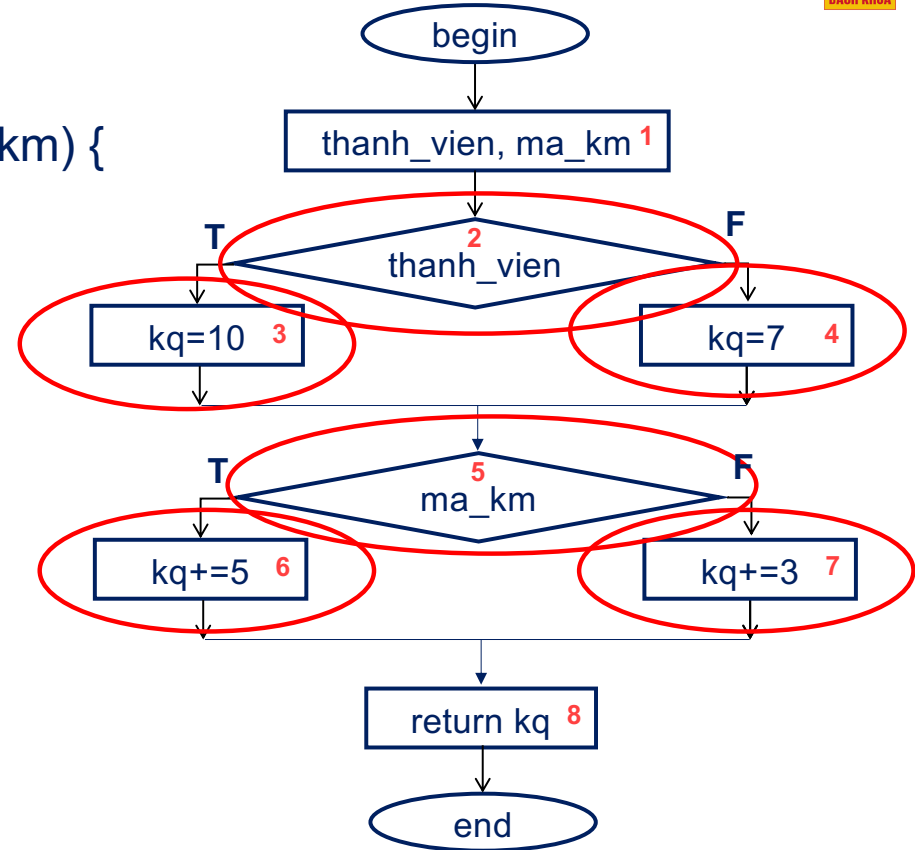
```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq = 0;  
    if (thanh_vien) kq = 10; else kq = 7;  
    if (ma_km) kq += 5; else kq += 3;  
    return kq;  
}
```

2. BÀI TẬP ÁP DỤNG



Đoạn mã chương trình

```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq;  
    if (thanh_vien) kq = 10; else kq = 7;  
    if (ma_km) kq += 5; else kq += 3;  
    return kq;  
}
```



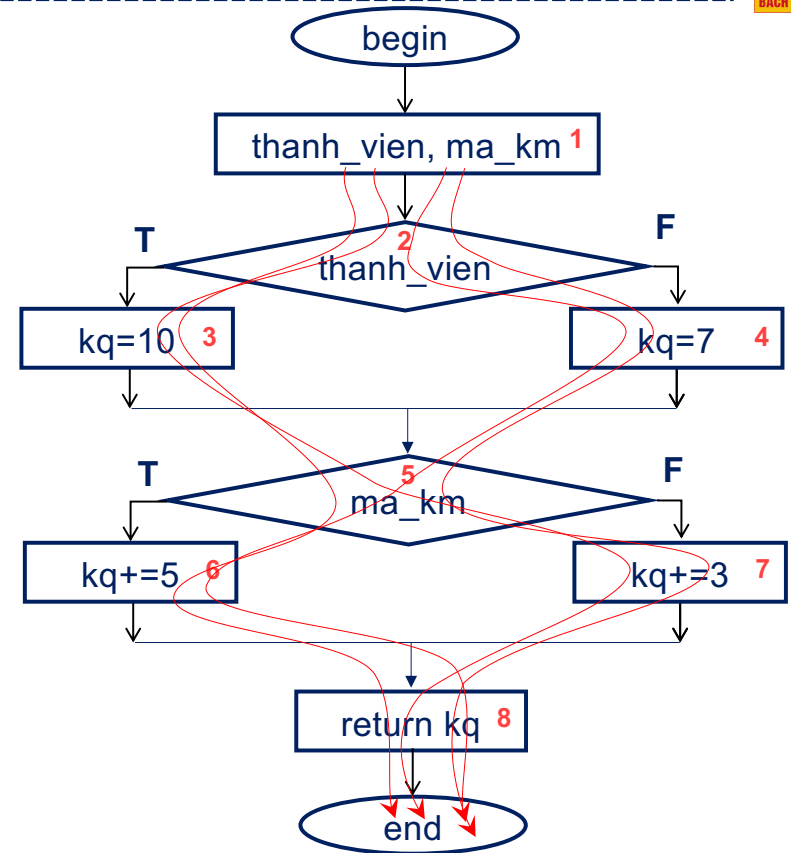
Hình 2.1: Sơ đồ luồng điều khiển – ví dụ 1

2. BÀI TẬP ÁP DỤNG



Phủ đường dẫn

- (1-2-3-5-6-8): **TC <T, T, 15>**
- (1-2-3-5-7-8): **TC <T, F, 13>**
- (1-2-4-5-6-8): **TC <F, T, 12>**
- (1-2-4-5-7-8): **TC <F, F, 10>**



Hình 2.1: Sơ đồ luồng điều khiển – ví dụ 1

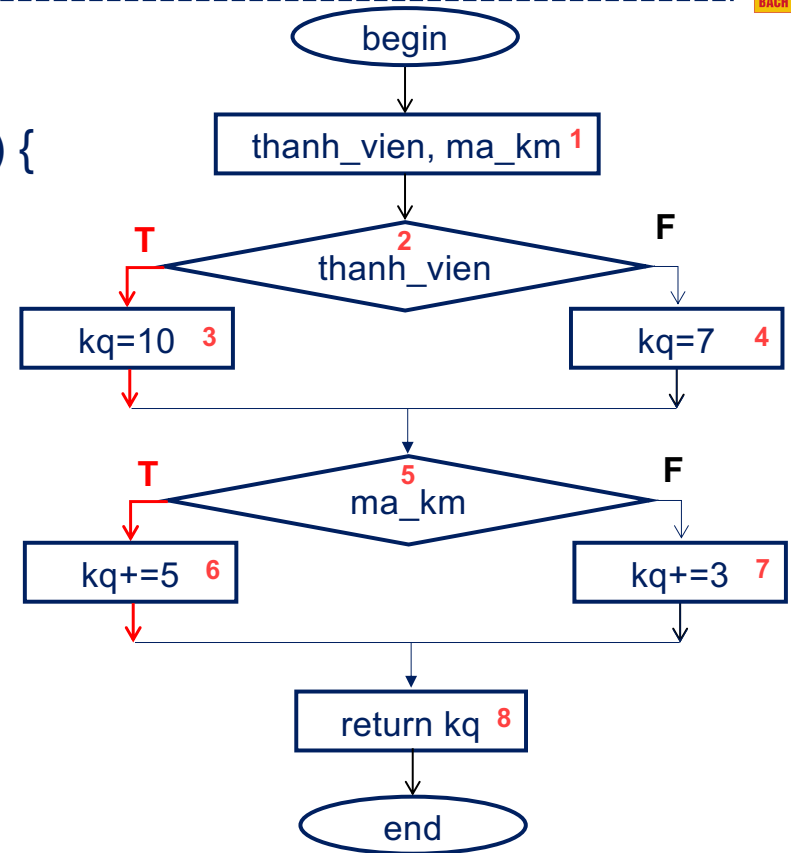
2. BÀI TẬP ÁP DỤNG



Bao phủ nhánh (1): T-T

```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq;  
    if (Tthanh_vien) kq = 10; else kq = 7;  
    if (Tma_km) kq += 5; else kq += 3;  
    return kq;  
}
```

▪ TC <T, T, 15>



Hình 2.2: Bao phủ nhánh T-T

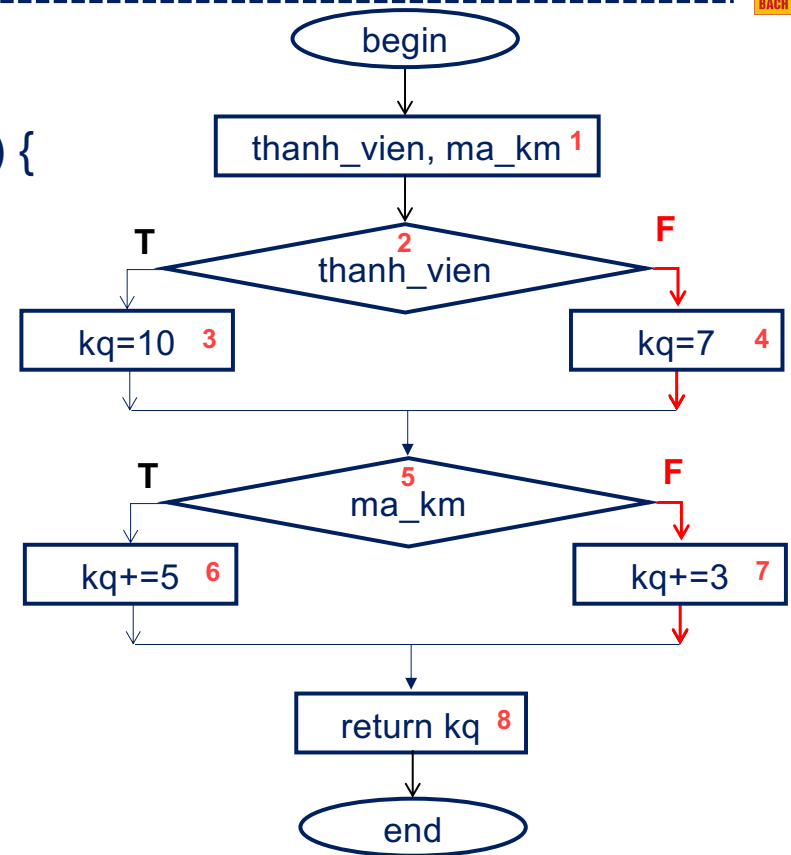
2. BÀI TẬP ÁP DỤNG



Bao phủ nhánh (2): **F-F**

```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq;  
    if (Fthanh_vien) kq = 10; else kq = 7;  
    if (Fma_km) kq += 5; else kq += 3;  
    return kq;  
}
```

▪ **TC <F, F, 10>**



Hình 2.3: Bao phủ nhánh F-F

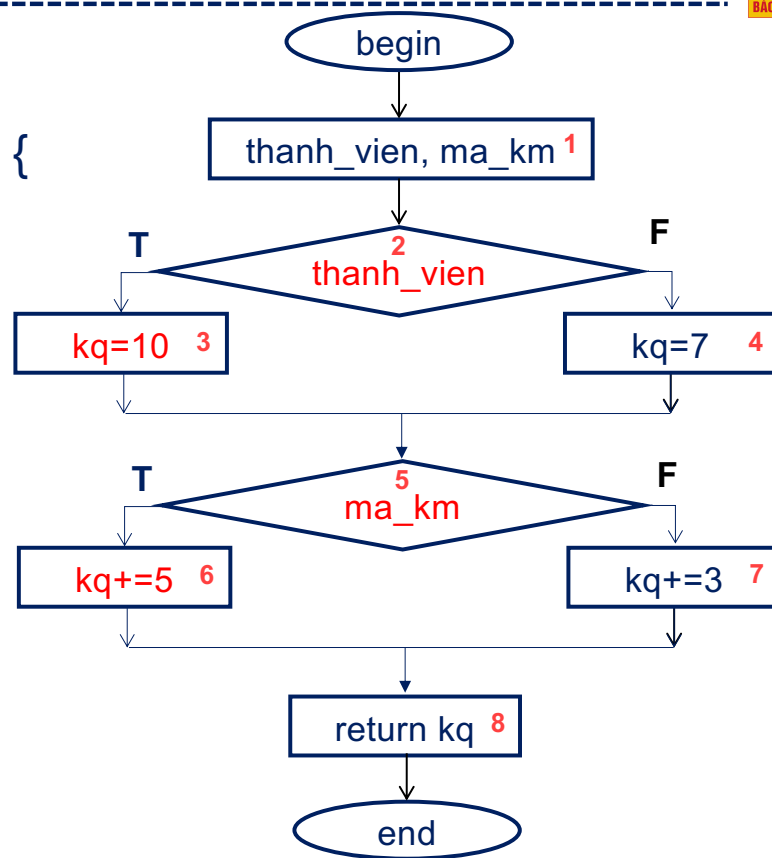
2. BÀI TẬP ÁP DỤNG



Bao phủ lệnh (1):

```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq;  
    if (thanh_vien) kq = 10; else kq = 7;  
    if (ma_km) kq += 5; else kq += 3;  
    return kq;  
}
```

▪ TC <T, T, 15>



Hình 2.4: Bao phủ lệnh `kq=10` và `kq+=5`

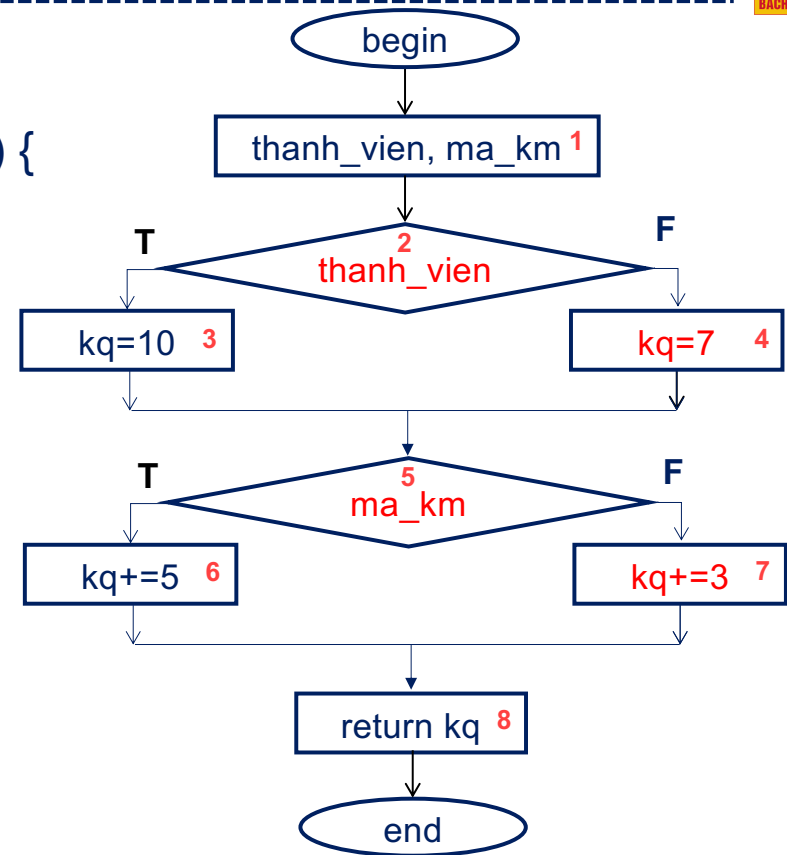
2. BÀI TẬP ÁP DỤNG



Bao phủ lệnh (2):

```
int giamgia(bool thanh_vien, bool ma_km) {  
    int kq;  
    if (thanh_vien) kq = 10; else kq = 7;  
    if (ma_km) kq += 5; else kq += 3;  
    return kq;  
}
```

▪ TC <F, F, 10>



Hình 2.5: Bao phủ lệnh `kq=7` và `kq+=3`

2. BÀI TẬP ÁP DỤNG



Nhận xét

- Trong bài tập này ta có
 - Bao phủ đường dẫn: 4 test case
 - Bao phủ nhánh: 2 test case
 - Bao phủ lệnh: 2 test case
- Ta thấy tiêu chí bao phủ đường dẫn cần đến 4 test case và là tiêu chí mạnh nhất.

2. BÀI TẬP ÁP DỤNG



Ví dụ 2: xét kết quả điểm học phần

Cho chương trình xét điểm học phần của sinh viên với mô tả như sau:

- Đầu vào:
 - điểm quá trình và điểm cuối kì (qt và ck) - giả thiết thuộc khoảng $[0, 10]$
- Đầu ra:
 - ‘đạt’ nếu $(qt * 0.3 + ck * 0.7) \geq 4$ và không có điểm liệt (< 3)
 - ‘không đạt’ nếu ngược lại

2. BÀI TẬP ÁP DỤNG



Đoạn mã chương trình

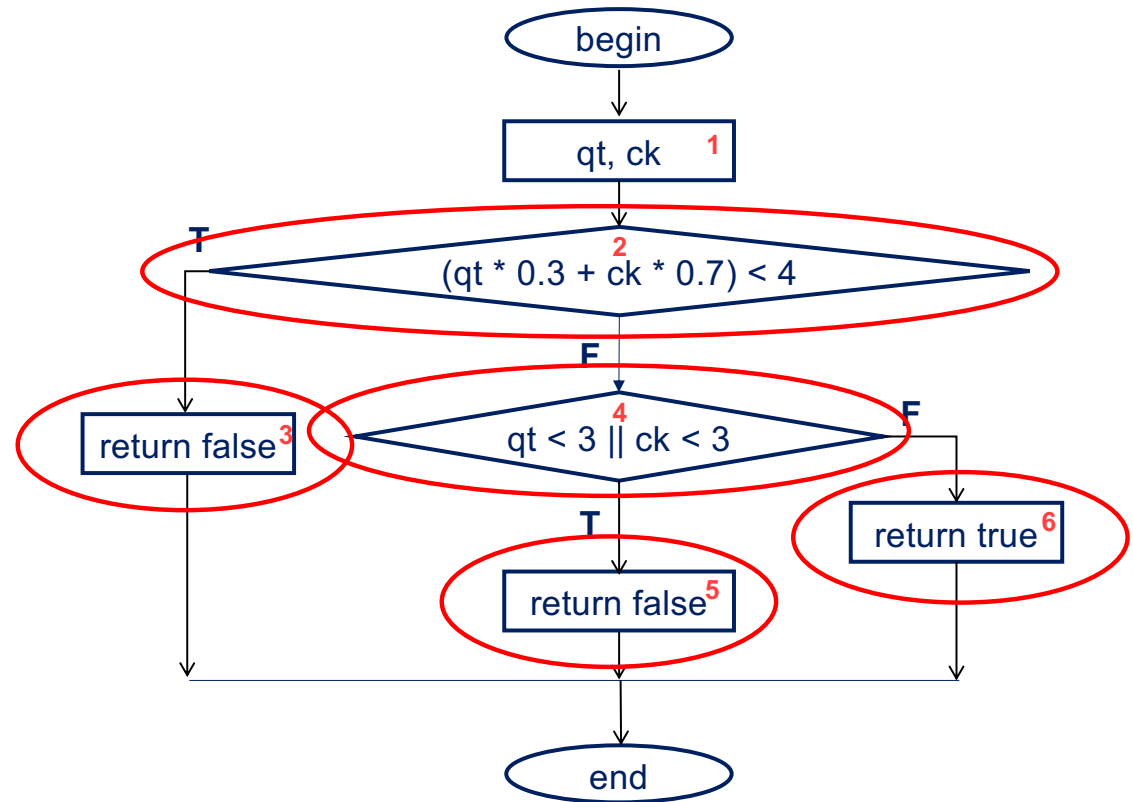
```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + ck * 0.7) < 4)  
        return false;  
    else  
        if (qt < 3 || ck < 3)  
            return false;  
        else  
            return true;  
}
```

2. BÀI TẬP ÁP DỤNG



Đoạn mã chương trình

```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + ck * 0.7) < 4)  
        return false;  
    else  
        if (qt < 3 || ck < 3)  
            return false;  
        else  
            return true;  
}
```



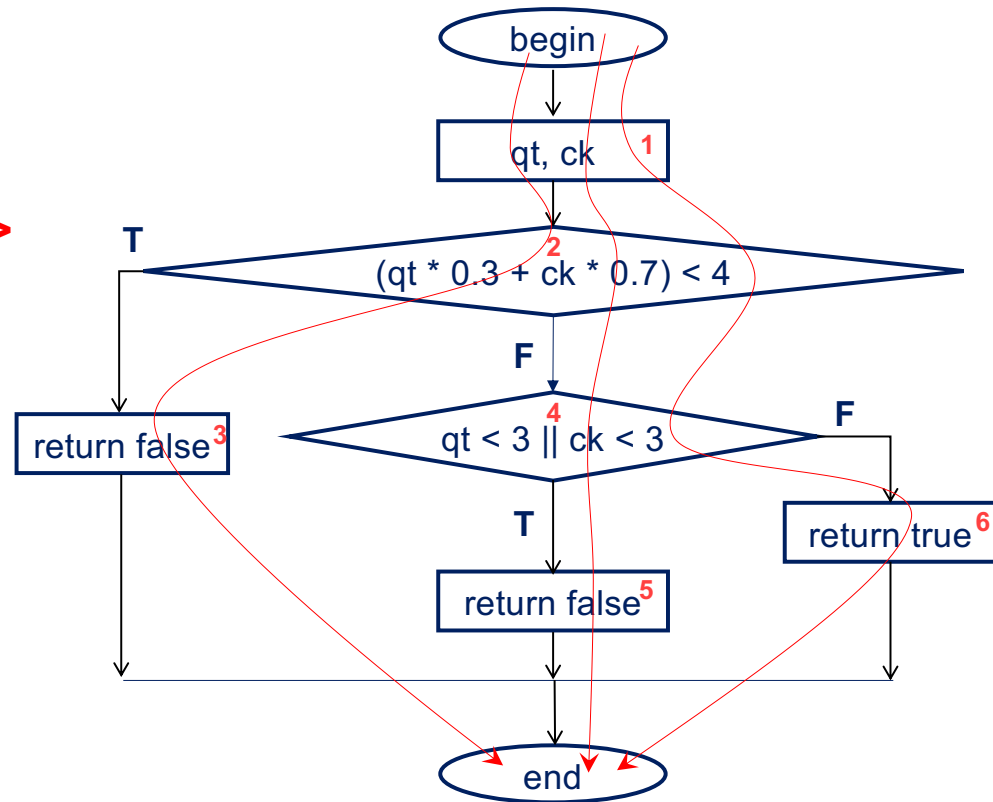
Hình 2.6: Sơ đồ luồng điều khiển – ví dụ 2

2. BÀI TẬP ÁP DỤNG



Phủ đường dẫn

- (1 – 2 – 3): **TC <1, 5, false>**
- (1 – 2 – 4 – 5): **TC <2, 8, false>**
- (1 – 2 – 4 – 6): **TC <4, 8, true>**



Hình 2.7: Sơ đồ luồng điều khiển – ví dụ 2

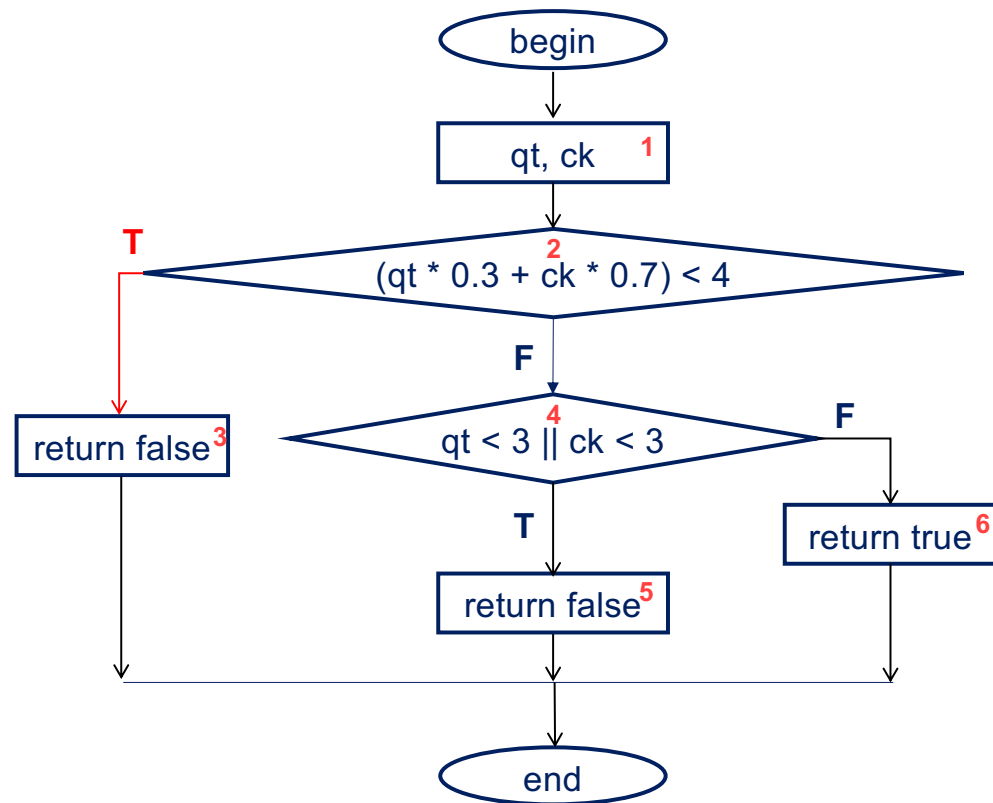
2. BÀI TẬP ÁP DỤNG



Bao phủ nhánh (1): **T**

```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + ckT * 0.7) < 4)  
        return false;  
    else  
        if (qt < 3 || ck < 3)  
            return false;  
        else  
            return true;  
}
```

TC <1, 5, false>



Hình 2.8: $(qt * 0.3 + ck * 0.7) < 4$ (T)

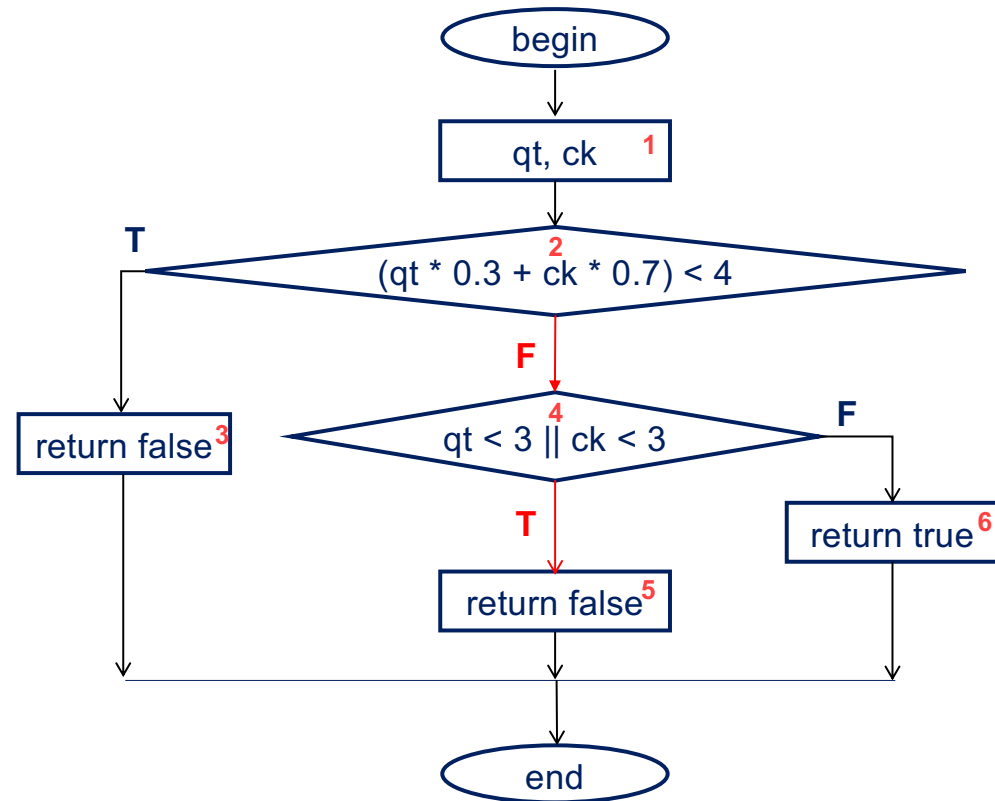
2. BÀI TẬP ÁP DỤNG



Bao phủ nhánh (2): F, T

```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + ckF * 0.7) < 4)  
        return false;  
    else  
        if (qt < 3 || ckT < 3)  
            return false;  
        else  
            return true;  
}
```

TC <2, 8, false>



Hình 2.9: $(qt * 0.3 + ck * 0.7) < 4$ (F) và $(qt < 3 \parallel ck < 3)$ (T)

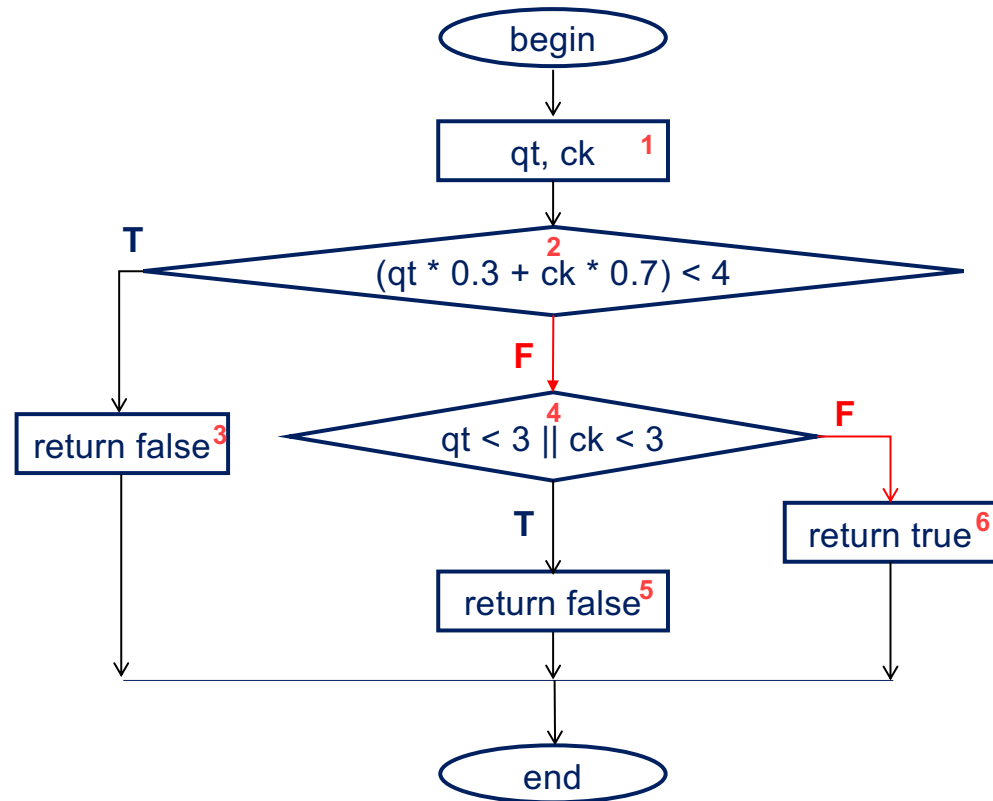
2. BÀI TẬP ÁP DỤNG



Bao phủ nhánh (3): F, **F**

```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + Fck * 0.7) < 4)  
        return false;  
    else  
        if (qt < 3 || Fck < 3)  
            return false;  
        else  
            return true;  
}
```

TC <4, 8, true>



Hình 2.10: $(qt * 0.3 + ck * 0.7) < 4$ (F) và $(qt < 3 \parallel ck < 3)$ (F) 81

2. BÀI TẬP ÁP DỤNG



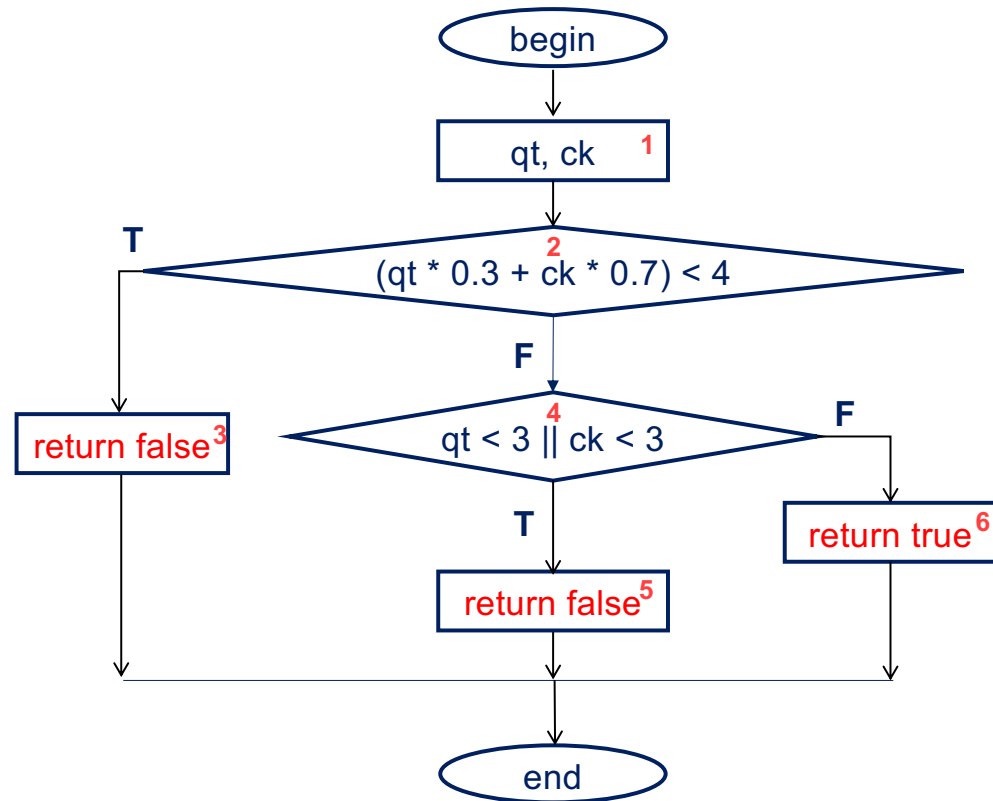
Bao phủ lệnh

```
bool kq(float qt, float ck) {  
    if ((qt * 0.3 + ck * 0.7) < 4)  
        return false; //1  
    else  
        if (qt < 3 || ck < 3)  
            return false; //2  
        else  
            return true; //3  
}
```

//1: TC <1, 5, false>

//2: TC <2, 8, false>

//3: TC <4, 8, true>



Hình 2.11: Bao phủ lệnh

2. BÀI TẬP ÁP DỤNG



Nhận xét

- Trong bài tập này cả 3 test case tìm được đều áp dụng với cùng 3 tiêu chí đã cho.

1. TC <1, 5, false>

2. TC <2, 8, false>

3. TC <4, 8, true>

- Như vậy không có sự phân biệt rõ ràng tiêu chí nào là mạnh nhất hay yếu nhất.

TỔNG KẾT VÀ GỢI MỞ



1. Bài tập vận dụng ở trên đã cung cấp cho người học cách thức giải quyết bài toán kiểm thử theo đồ thị luồng điều khiển.
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về các kĩ thuật kiểm thử hộp đen.

Phương pháp kiểm thử hộp đen

NỘI DUNG



1. Kiểm thử hộp đen
2. Kỹ thuật phân lớp tương đương

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về phương pháp kiểm thử hộp đen
2. Nắm được kỹ thuật phân lớp tương đương

NỘI DUNG TIẾP THEO



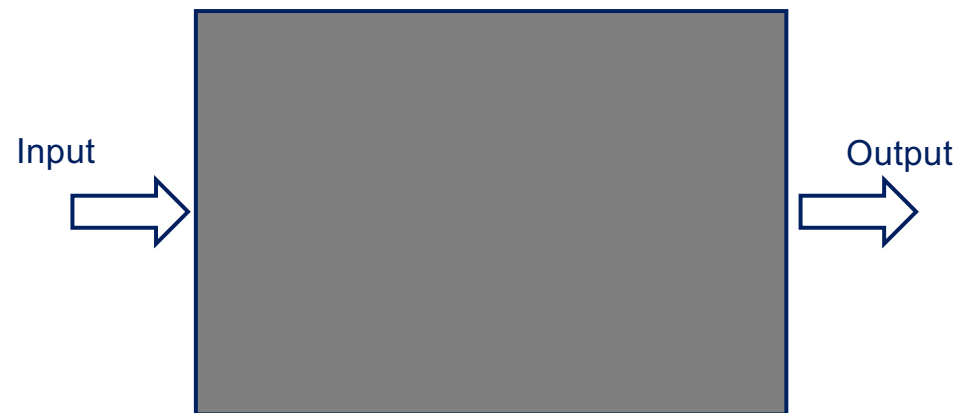
1. Kiểm thử hộp đen

2. Kỹ thuật phân lớp tương đương

1. KIỂM THỬ HỘP ĐEN



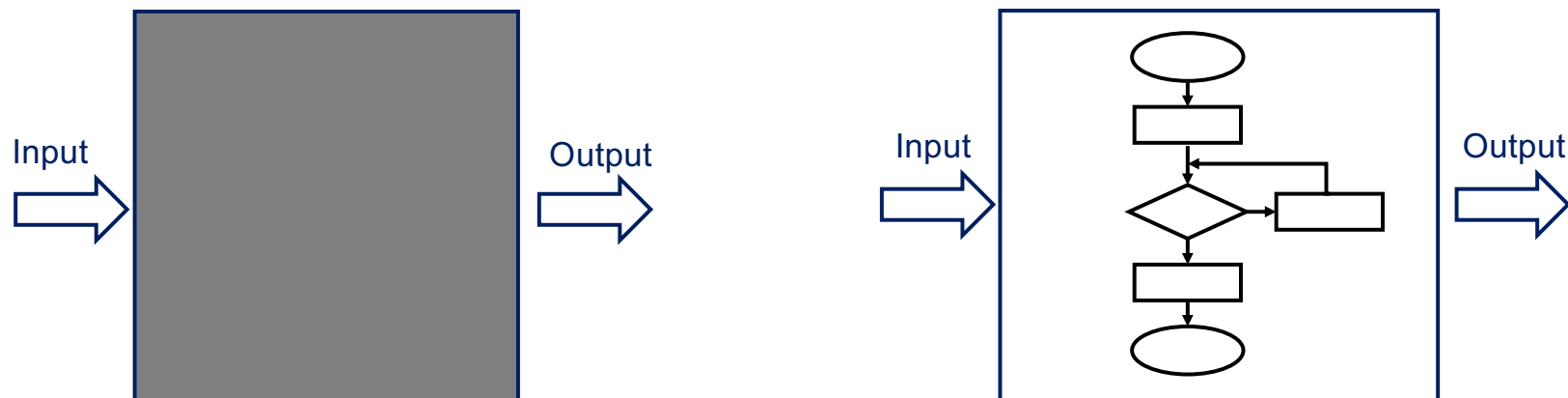
- Kiểm thử hộp đen là chiến lược thực hiện dựa trên **yêu cầu và đặc tả của chức năng**, trong đó xác minh đầu ra của chức năng mà **không quan tâm tới cấu trúc bên trong chương trình**.



Hình 1.1: Kiểm thử hộp đen

1. KIỂM THỬ HỘP ĐEN

- Sẽ **không biết lỗi được kiểm soát bên trong như thế nào**, cũng không thể đảm bảo kiểm thử được tất cả các đường dẫn như các chiến lược kiểm thử hộp trắng
- Kiểm thử hộp đen được coi là **kiểm thử chức năng** trong khi kiểm thử hộp trắng là **kiểm thử cấu trúc**.

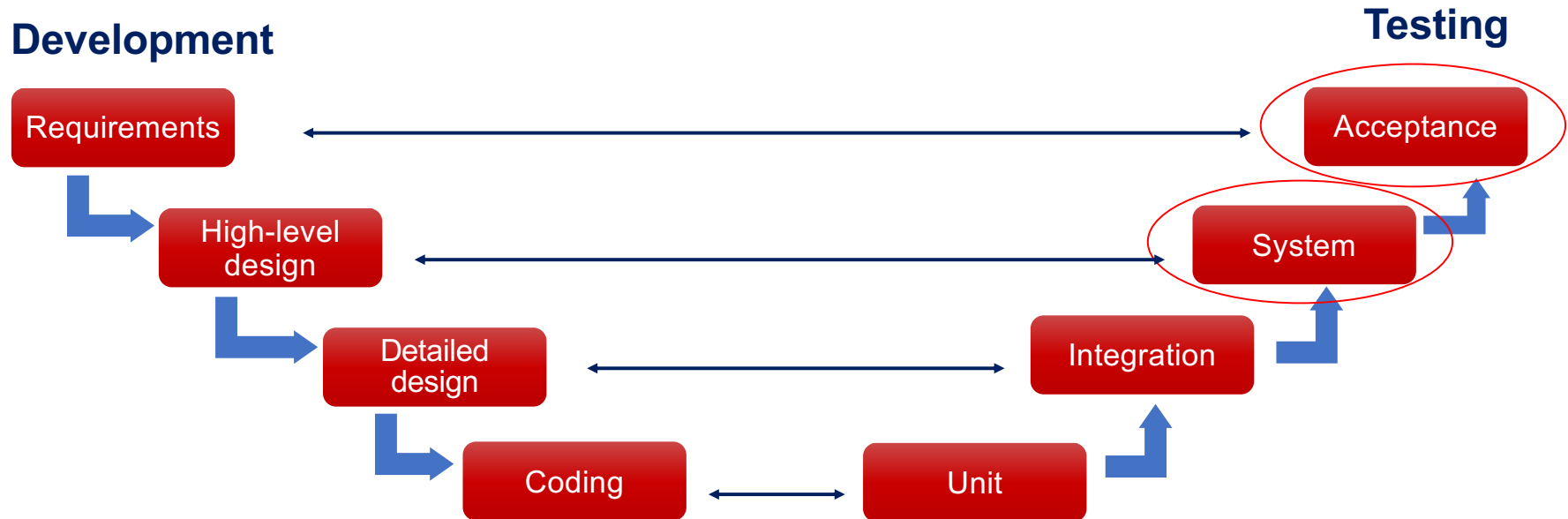


Hình 1.2: Kiểm thử hộp đen và kiểm thử hộp trắng

1. KIỂM THỬ HỘP ĐEN



- Thường được thực hiện bởi các kiểm thử viên, kĩ sư đảm bảo chất lượng
- Được sử dụng ở giai đoạn kiểm thử hệ thống và kiểm thử chấp nhận



Hình 1.3: Mô hình chữ V và kiểm thử hộp đen

1. KIỂM THỬ HỘP ĐEN



- Đầu vào của kiểm thử hộp đen là những **đặc tả chi tiết về chức năng**
- Đầu ra là các **kết quả hoạt động** của hệ thống tương ứng với từng loại dữ liệu đầu vào và kịch bản kiểm thử
- Các phương pháp thiết kế ca kiểm thử:
 - Phân chia lớp tương đương
 - Phân tích giá trị biên
 - Sử dụng bảng quyết định
 - Kiểm thử trạng thái

NỘI DUNG TIẾP THEO



1. Kiểm thử hộp đen

2. Kỹ thuật phân lớp tương đương

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG

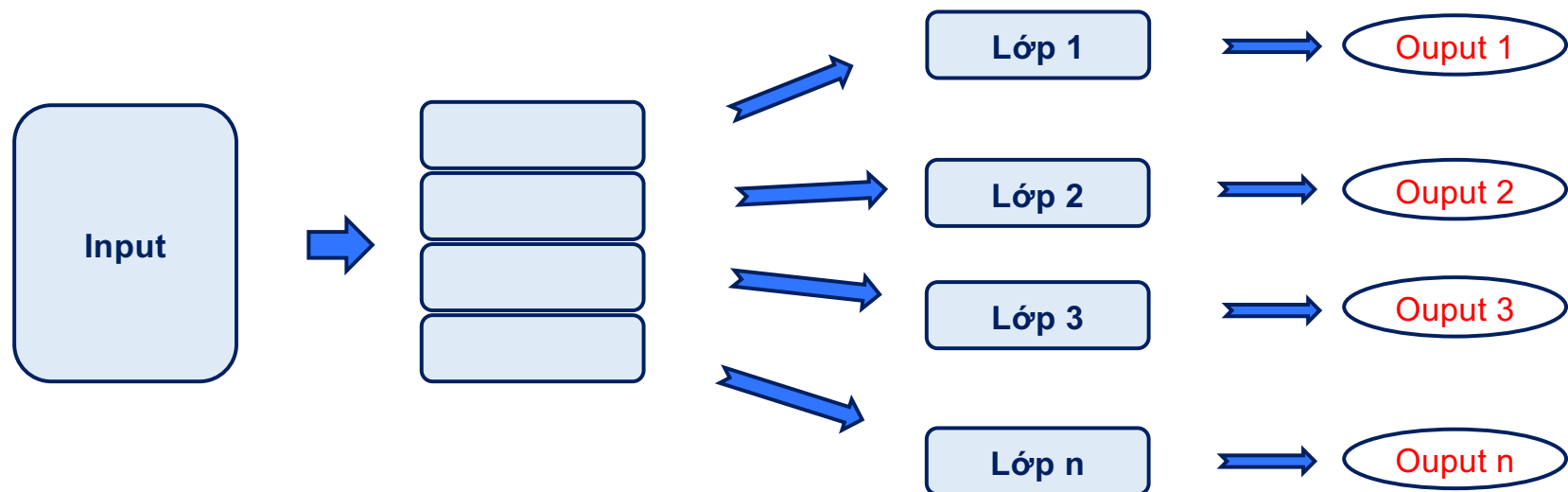


- Dựa vào **miền dữ liệu trong đặc tả** của đầu vào (input) và đầu ra (output) để xác định các phân lớp tương đương
- Một phân lớp tương đương được định nghĩa là một **miền dữ liệu** (input hoặc output) trong đó với **mọi dữ liệu thuộc miền này thì chức năng được kiểm thử sẽ thực hiện cùng một hành vi** (kết quả giống nhau)
- Ý tưởng: có thể kiểm thử một giá trị đại diện trong vùng để tiết kiệm thời gian

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



- Cần thực hiện **phân chia miền đầu vào thành các lớp dữ liệu**.
- Một lớp tương đương: **mọi giá trị dữ liệu trong lớp đều tạo cho hệ thống các hoạt động tương đương nhau**.



Hình 2.1: Kỹ thuật phân lớp tương đương

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



- Có thể phân dữ liệu thành hai lớp
 - Lớp hợp lệ
 - Lớp không hợp lệ
- **Ví dụ 1: Giá trị x chỉ có thể dao động từ 0 đến 10**
 - Lớp hợp lệ là $0 \leq x \leq 10$
 - Hai lớp không hợp lệ là: $x < 0$ và $x > 10$

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



- **Ví dụ 2: Các loại xe được đăng ký là xe bus, xe tải, xe taxi và xe máy**
 - Các lớp hợp lệ tương ứng với các loại xe: “xe bus”, “xe tải”, “xe taxi” và “xe máy”
 - Lớp không hợp lệ là: loại xe khác ví dụ như “xe đạp”

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



- Ví dụ 3: trường password được giới hạn từ 6-10 ký tự

Mô tả	Kết quả
0 đến 5 ký tự	Hệ thống không chấp nhận
6 đến 10 ký tự	Hệ thống chấp nhận
11 đến 14 ký tự	Hệ thống không chấp nhận

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Ví dụ 4: Kiểm tra số nguyên âm hay dương

- Hàm thực hiện kiểm tra một số nguyên là âm hay dương.
- Phân tích đầu vào và kết quả:
 - Input: một số nguyên
 - Output: là *true* nếu là số nguyên âm, là *false* nếu là số nguyên dương

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG

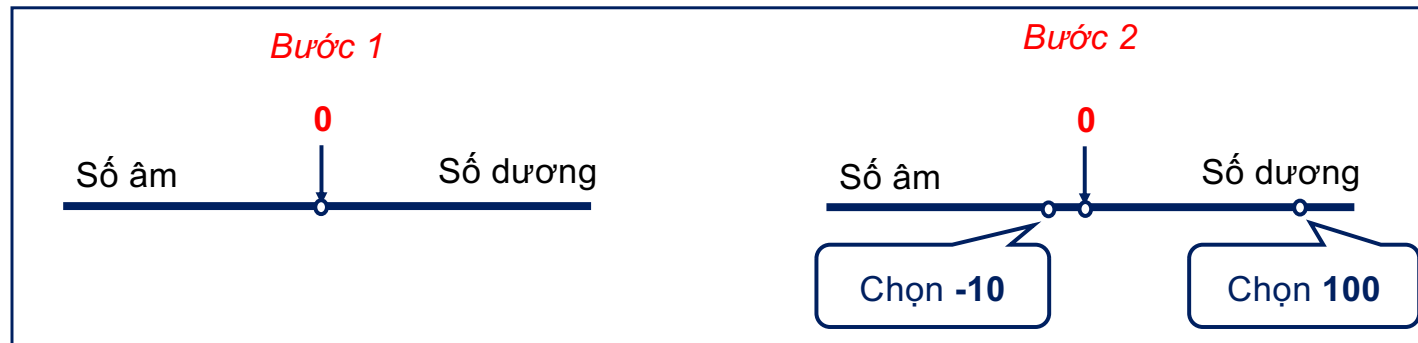


Ví dụ 4: Kiểm tra số nguyên âm hay dương

▪ Cách giải quyết bài toán

- Bước 1: Xác định 2 lớp tương đương ứng với $\text{input} > 0$ (dương) và $\text{input} < 0$ (âm)
- Bước 2: Ở mỗi phân lớp tương đương lấy 1 giá trị đại diện để xây dựng ca kiểm thử.

Ta có 2 test case: **<-10, true>** và **<100, false>**



Hình 2.2: Các bước thiết kế test case

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Ví dụ 5: Hệ thống xác nhận đăng ký hồ sơ bảo hiểm

- Đặc tả:
 - Hệ thống sẽ từ chối hoặc chấp nhận hồ sơ đăng ký bảo hiểm
 - Từ chối hồ sơ nam trên 80 tuổi;
 - Từ chối hồ sơ nữ trên 85 tuổi.
- Cách thức xác định các lớp tương đương như thế nào?

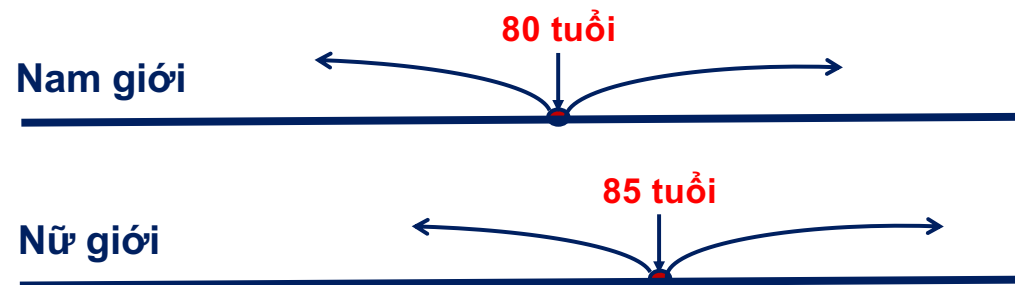
2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Ví dụ 5: Hệ thống xác nhận đăng ký hồ sơ bảo hiểm

▪ Phân tích bài toán

- Input: Giới tính & Tuổi;
- Output: Chấp nhận/Từ chối;



Hình 2.3: Phân chia các lớp tương đương

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Ví dụ 5: Hệ thống xác nhận đăng ký hồ sơ bảo hiểm

	Lớp
Nam giới	C1: Input: Nam, trên 80 tuổi
	C2: Input: Nam, nhỏ hơn hoặc bằng 80 tuổi
Nữ giới	C3: Input: Nữ, trên 85 tuổi
	C4: Input: Nữ, nhỏ hơn hoặc bằng 85 tuổi

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Nhận xét

▪ Ưu điểm

- Giảm số lượng ca kiểm thử (không phải xem xét toàn bộ miền dữ liệu);
- Đảm bảo mỗi một lớp đều được kiểm thử.
- Giảm thời gian kiểm thử

2. KỸ THUẬT PHÂN LỚP TƯƠNG ĐƯƠNG



Nhận xét

▪ Nhược điểm

- Phân chia lớp tương đương phụ thuộc vào kinh nghiệm của kiểm thử viên
 - Có thể bỏ qua các phân lớp dữ liệu không hợp lệ (invalid data)
- Không xem xét các giá trị tại biên của các lớp
 - Nên kết hợp với phân tích giá trị biên để xây dựng các ca kiểm thử
- Khó phân chia rõ ràng các lớp tương đương khi dữ liệu có mối quan hệ với nhau và không còn độc lập
 - Kết hợp với bảng quyết định để xây dựng các ca kiểm thử

TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học một số **khái niệm cơ bản** về phương pháp kiểm thử hộp đen, kỹ thuật phân lớp tương đương
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về quy trình đảm bảo chất lượng phần mềm và bảo trì phần mềm

Bài tập: Phương pháp kiểm thử hộp đen

NỘI DUNG



1. Bài toán kiểm thử với lớp tương đương
2. Bài tập áp dụng

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về **phương pháp** kiểm thử hộp đen với kỹ thuật phân lớp tương đương
2. Biết **vận dụng** kiến thức về kỹ thuật kiểm thử phân lớp tương đương cho bài toán kiểm thử

NỘI DUNG TIẾP THEO



1. Bài toán kiểm thử với lớp tương đương

2. Bài tập áp dụng

1. BÀI TOÁN KIỂM THỬ LỚP TƯƠNG DƯƠNG



Lớp tương đương (nhắc lại)

- Dựa vào miền dữ liệu trong đặc tả của đầu vào (input) và đầu ra (output) để **xác định các lớp tương đương**
- Một lớp tương đương
 - Bao gồm một **tập hợp dữ liệu** làm cho hệ thống hoạt động theo cùng **một cách** (gọi là hành vi)
 - Mọi giá trị dữ liệu trong lớp đều **tương đương dưới góc độ hành vi** đó

1. BÀI TOÁN KIỂM THỬ LỚP TƯƠNG DƯƠNG



Bài toán kiểm thử với lớp tương đương

- Cho mô tả bài toán, các dữ liệu đầu vào, miền dữ liệu đầu vào và các yêu cầu về kết quả đầu ra.
- Yêu cầu:
 - Thiết kế các ca kiểm thử tương ứng theo kĩ thuật phân lớp tương đương.

1. BÀI TOÁN KIỂM THỬ LỚP TƯƠNG DƯƠNG



- Cách giải quyết bài toán
 - Bước 1: Xác định các dữ liệu đầu vào và miền kết quả tương ứng.
 - Bước 2: Phân lớp cho miền giá trị của dữ liệu đầu vào.
 - Bước 3: Lấy giá trị cho các dữ liệu đầu vào dựa trên các lớp đã được phân ở bước 2.
 - Bước 4: Xác định các test case tương ứng với dữ liệu ở bước 3.

NỘI DUNG TIẾP THEO



1. Bài toán kiểm thử với lớp tương đương

2. Bài tập áp dụng

2. BÀI TẬP ÁP DỤNG



Ví dụ 1: Mua vé xe buýt

- Mô tả:
 - Xe buýt hoạt động hằng ngày bắt đầu từ 5:00 đến 22:00. Mua vé thường vào giờ cao điểm từ 5:00 đến 9:30 hoặc từ 16:00 đến 19:30. Mua vé tiết kiệm cho xe sau 9:30 đến trước 16:00 và sau 19:30.
- Yêu cầu:
 - Liệt kê ra các lớp tương đương, các vùng hợp lệ và không hợp lệ.
 - Thiết kế các test case để dựa vào các vùng tương đương ở trên.

2. BÀI TẬP ÁP DỤNG



Ví dụ 1: Mua vé xe buýt

- **Các lớp:** (**0h–4h59**), (**5h–9h30**), (**9h31–15h59**), (**16h–19h30**), (**19h31–22h**), (**22h01–23h59**)
- **Vé thường:** Các vùng hợp lệ: (**5h–9h30**), (**16h–19h30**); các vùng không hợp lệ: (**0h–4h59**), (**9h31–15h59**), (**22h01–23h59**)
- **Vé tiết kiệm:** Các vùng hợp lệ: (**9h31–15h59**), (**19h31–22h**); các vùng không hợp lệ: (**0h–4h59**), (**5h–9h30**), (**16h–19h30**), (**22h01–23h59**)

2. BÀI TẬP ÁP DỤNG



Ví dụ 1: Mua vé xe buýt

- **Các lớp:** (**0h–4h59**), (**5h–9h30**), (**9h31–15h59**), (**16h–19h30**), (**19h31–22h**), (**22h01–23h59**)
- **Vé thường:** Các vùng hợp lệ: (**5h–9h30**), (**16h–19h30**); các vùng không hợp lệ: (**0h–4h59**), (**9h31–15h59**), (**22h01–23h59**)
- **Vé tiết kiệm:** Các vùng hợp lệ: (**9h31–15h59**), (**19h31–22h**); các vùng không hợp lệ: (**0h–4h59**), (**5h–9h30**), (**16h–19h30**), (**22h01–23h59**)

2. BÀI TẬP ÁP DỤNG



Ví dụ 1: Mua vé xe buýt

- Bảng liệt kê các lớp tương đương

Thời gian	00:00-04:59	05:00-09:30	09:31-15:59	16:00-19:30	19:31-22:00	22:01-23:59
Loại vé	Invalid	Thường	Tiết kiệm	Thường	Tiết kiệm	Invalid

- Các test case
 - TC1: <04:00, Invalid>; TC2: <06:00, Thường>; TC3: <10:00, Tiết kiệm>; TC4: <17:00, Thường>; TC5: <20:00, Tiết kiệm>; TC6: <23:00, Invalid>

2. BÀI TẬP ÁP DỤNG



Ví dụ 2: Xét điểm thi của sinh viên

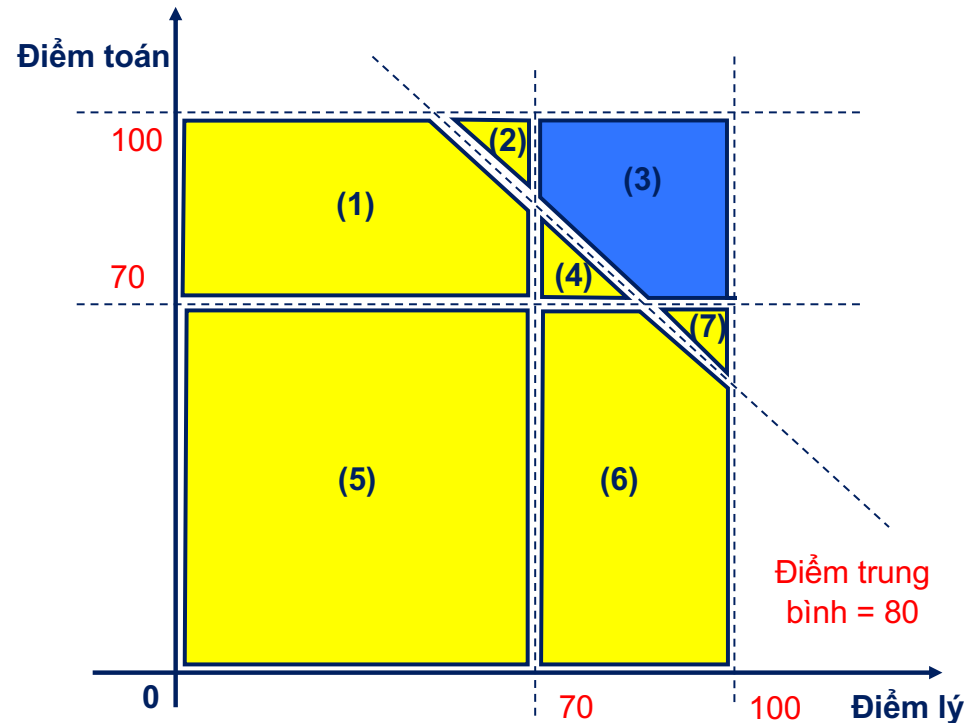
- Chương trình nhận đầu vào là giá trị điểm của hai môn toán và vật lý.
- Sinh viên sẽ vượt qua kì thi nếu thỏa mãn:
 1. Cả điểm toán và điểm vật lý ≥ 70 (thang điểm 100)
 2. Điểm trung bình của toán và vật lý ≥ 80
- Sinh viên không đạt trong kì thi nếu có kết quả ngược lại.

2. BÀI TẬP ỨNG DỤNG



Xác định các lớp

- Giới hạn bởi các giá trị
 - Toán = 100
 - Toán = 70
 - Lý = 100
 - Lý = 70
 - $(\text{Toán} + \text{Lý})/2 = 80$



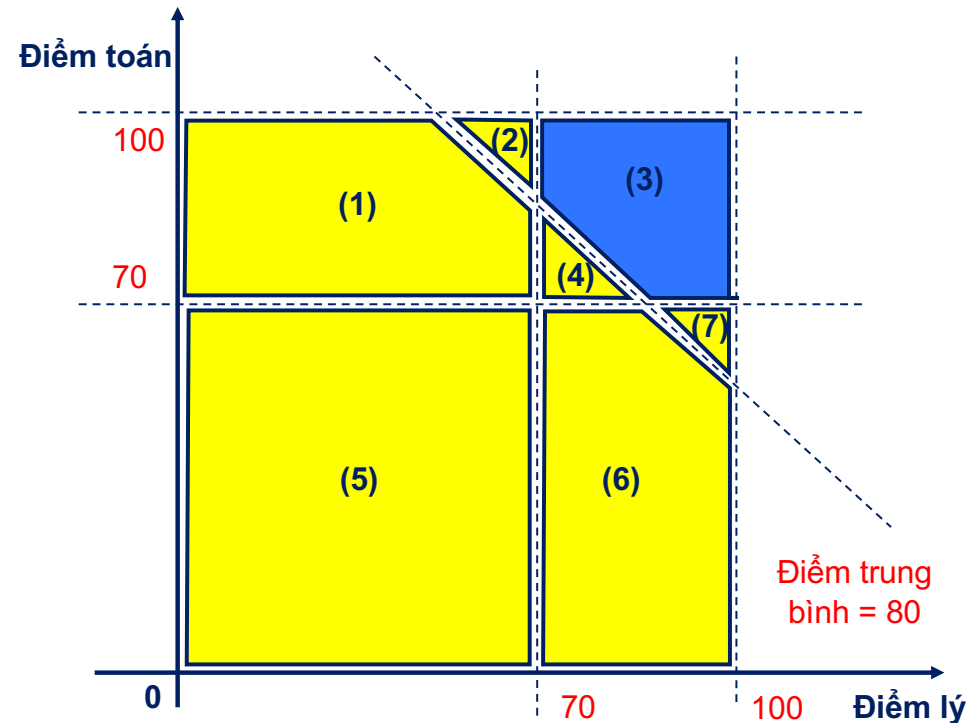
Hình 2.1: Xác định các lớp

2. BÀI TẬP ÁP DỤNG



Xác định các test case

TC	Toán	Lý	Kết quả
1	80	50	Không đạt
2	96	66	Không đạt
3	86	86	Đạt
4	75	75	Không đạt
5	50	50	Không đạt
6	50	80	Không đạt
7	66	96	Không đạt



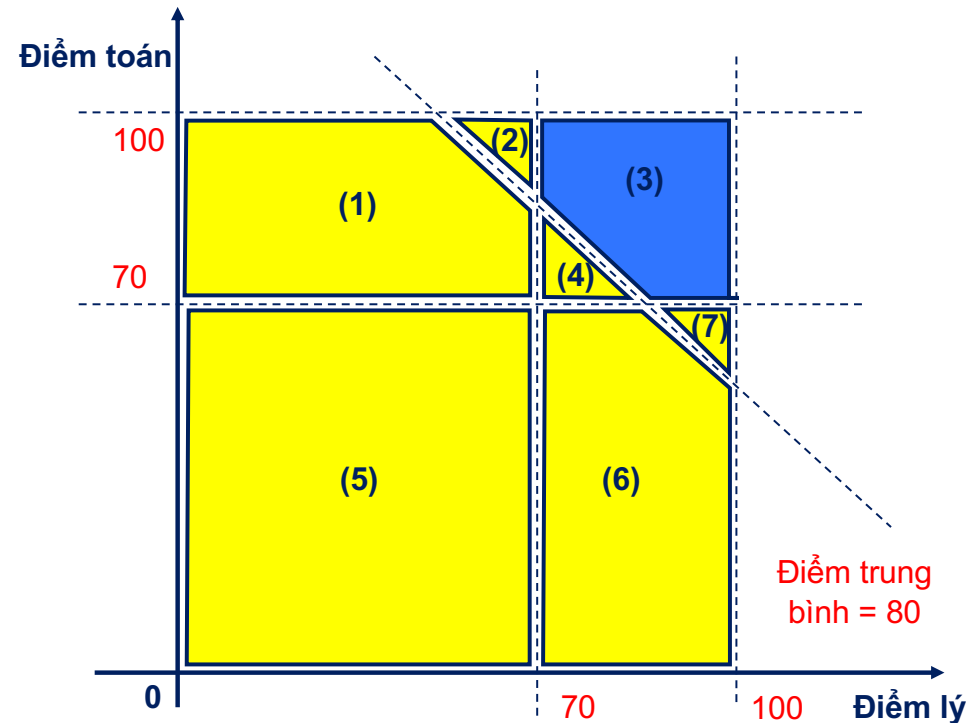
Hình 2.1: Xác định các lớp

2. BÀI TẬP ÁP DỤNG



Câu hỏi

- Trong trường hợp ngoài điểm toán, điểm lý còn có thêm điểm hóa thì bài toán sẽ được giải quyết như thế nào?
- Nhận xét: Bài toán sẽ phức tạp hơn nếu số lượng lớp tương đương tăng lên!



Hình 2.1: Xác định các lớp

TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học cách thức cơ bản để giải quyết bài toán kiểm thử hộp đen với kỹ thuật phân lớp tương đương
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về quy trình đảm bảo chất lượng phần mềm và bảo trì phần mềm

Quy trình đảm bảo chất lượng phần mềm

NỘI DUNG



1. Các vấn đề trong đảm bảo chất lượng phần mềm
2. Quy trình đảm bảo chất lượng phần mềm

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về các **vấn đề** trong đảm bảo chất lượng phần mềm
2. Nắm được **quy trình** đảm bảo chất lượng phần mềm

NỘI DUNG TIẾP THEO



1. Các vấn đề trong đảm bảo chất lượng phần mềm

2. Quy trình đảm bảo chất lượng phần mềm

1. CÁC VẤN ĐỀ TRONG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Chất lượng phần mềm

- Chất lượng phần mềm được định nghĩa là
 - Mức độ hệ thống hay thành phần **đạt được các yêu cầu** đặt ra,
 - Mức độ hệ thống hay thành phần **đạt được những mong đợi của khách hàng hay người dùng** hệ thống đó
- Chất lượng được nhìn nhận theo 2 quan điểm
 - Quan điểm của người phát triển
 - Quan điểm của người dùng

1. CÁC VẤN ĐỀ TRONG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Các vấn đề trong đảm bảo chất lượng phần mềm

- Quan điểm về chất lượng của người dùng và đội phát triển khác nhau
 - Đội phát triển quan tâm đến khía cạnh phát triển hệ thống: tái sử dụng, dễ kiểm thử, dễ bảo trì. Người dùng quan tâm đến: hiệu quả, hiệu suất và độ tin cậy của phần mềm
 - Nhưng đôi khi để đảm bảo cho việc dễ tái sử dụng, dễ bảo trì các tiêu chí chất lượng khác như hiệu quả và độ tin cậy có thể giảm đi
- Đặc tả phần mềm không bao giờ là đầy đủ và nhất quán nên rất khó để quy chuẩn được chất lượng phần mềm ngay từ đầu

NỘI DUNG TIẾP THEO



1. Các vấn đề trong đảm bảo chất lượng phần mềm

2. Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



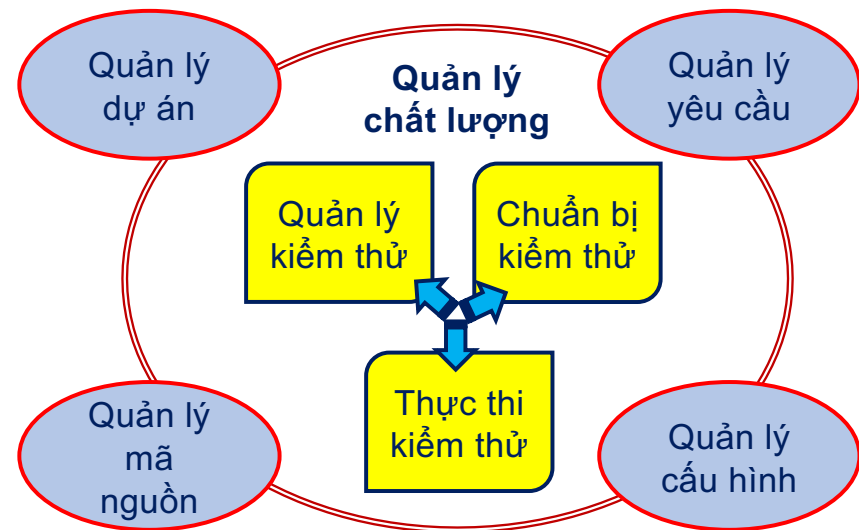
- Quy trình đảm bảo chất lượng được thực hiện để xác định xem sản phẩm hoặc dịch vụ đang được phát triển có **đạt được các yêu cầu** theo đặc tả ban đầu hay không
- Quy trình đảm bảo chất lượng là một **quy trình mang tính hệ thống**

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



Cần được **tích hợp** với bất kỳ quy trình nào khác trong vòng đời phát triển của hệ thống phần mềm

- Quy trình **quản lý dự án**
- Quy trình **quản lý yêu cầu**
- Quy trình **quản lý cấu hình**
- Quy trình **quản lý mã nguồn**



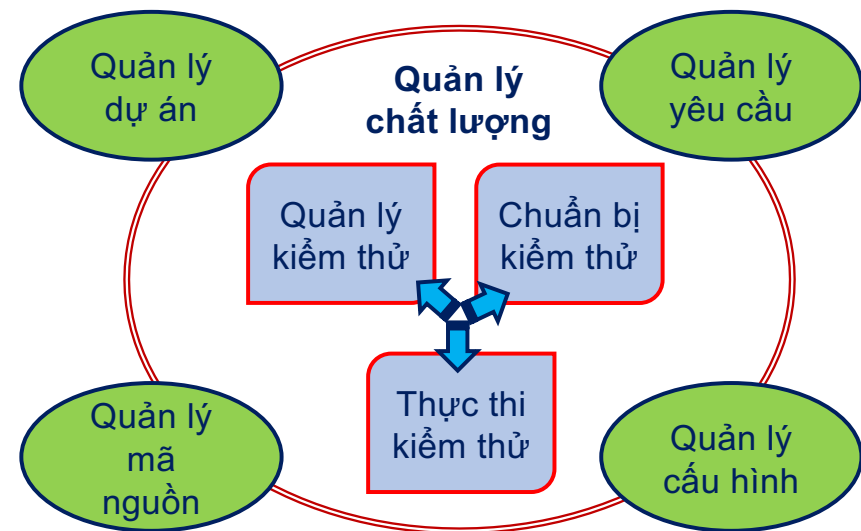
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



Quy trình **kiểm thử** là nền tảng của quy trình **đảm bảo chất lượng phần mềm** bao gồm 3 hoạt động chính

- Quản lý kiểm thử
- Chuẩn bị kiểm thử/Kế hoạch kiểm thử
- Thực thi kiểm thử



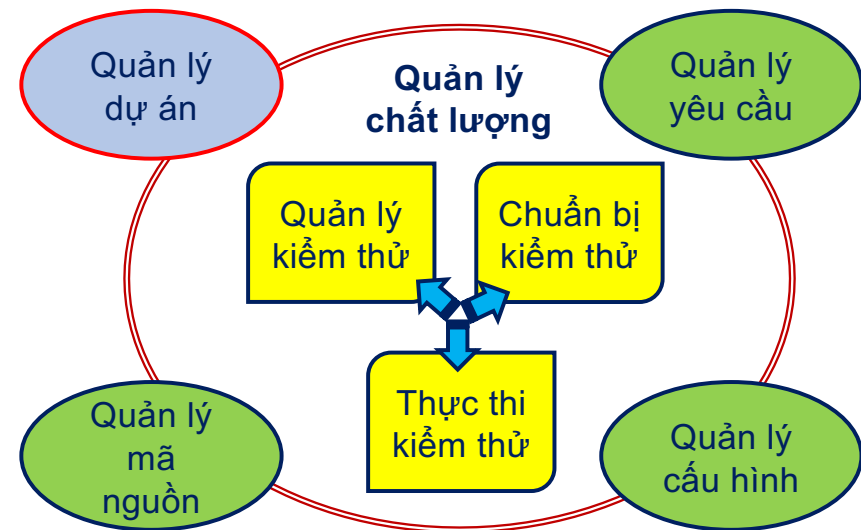
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Quản lý dự án

- Xác định các **mục tiêu** và **tiêu chí** chất lượng cần quan tâm
- Xác định các **thủ tục kiểm thử** và đảm bảo chất lượng
- Chỉ định trước **các nguồn lực** cần thiết



Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

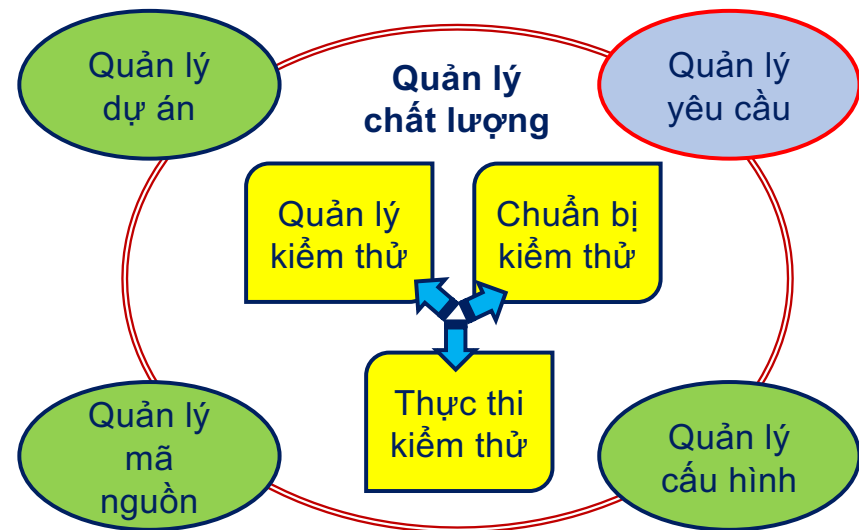
2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Quản lý yêu cầu

Cần đảm bảo các yêu cầu cho sản phẩm được tổ chức một cách có hệ thống.

- Xác định tất cả **các chức năng** của hệ thống
- Xác định **các ngoại lệ** từ các yêu cầu
- Đảm bảo đội dự án nắm được **các vấn đề**
- Cung cấp cơ sở cho **các ước tính, phát triển mã nguồn, và quy trình kiểm chứng**



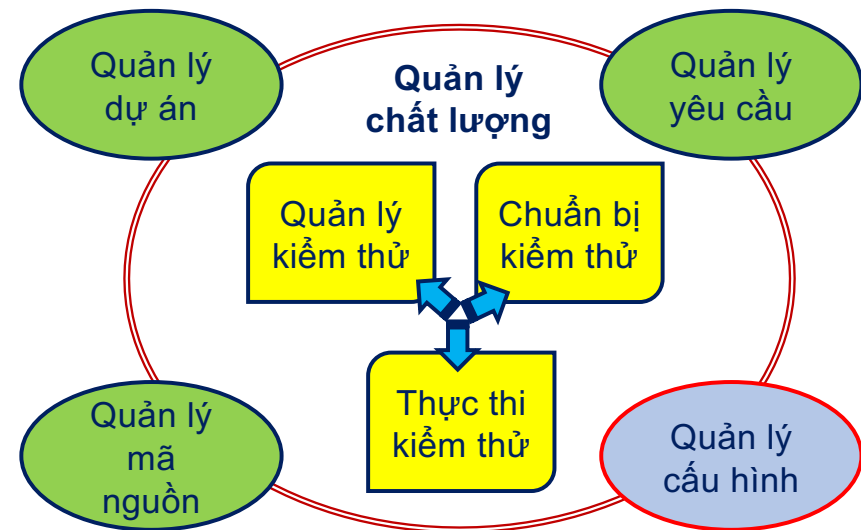
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Quản lý cấu hình

- Quản lý cấu hình phần mềm là **bắt buộc** trong mọi dự án phát triển phần mềm.
- Đảm bảo chất lượng phần mềm yêu cầu phải quản lý cấu hình một cách **chặt chẽ** và **tuân theo** các quy trình cụ thể.



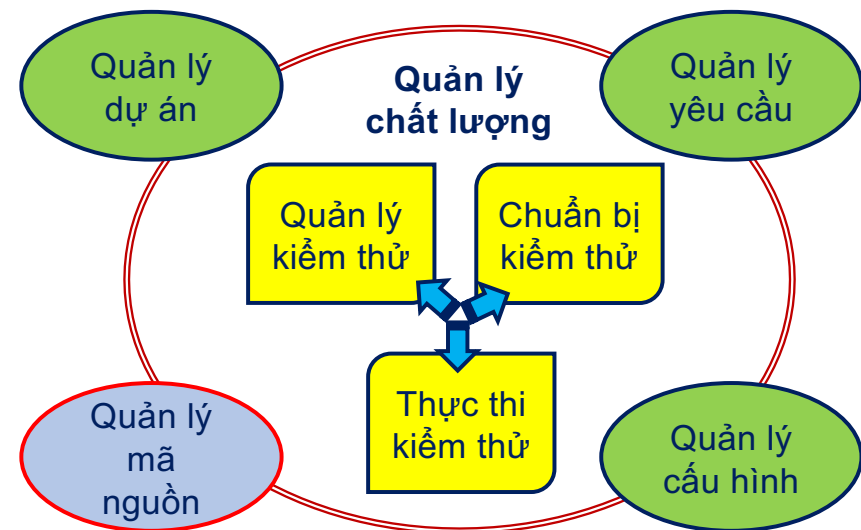
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Quản lý mã nguồn

- Áp dụng quy trình vào phát triển mã nguồn sẽ làm **tăng hiệu quả** của quy trình đảm bảo chất lượng phần mềm.



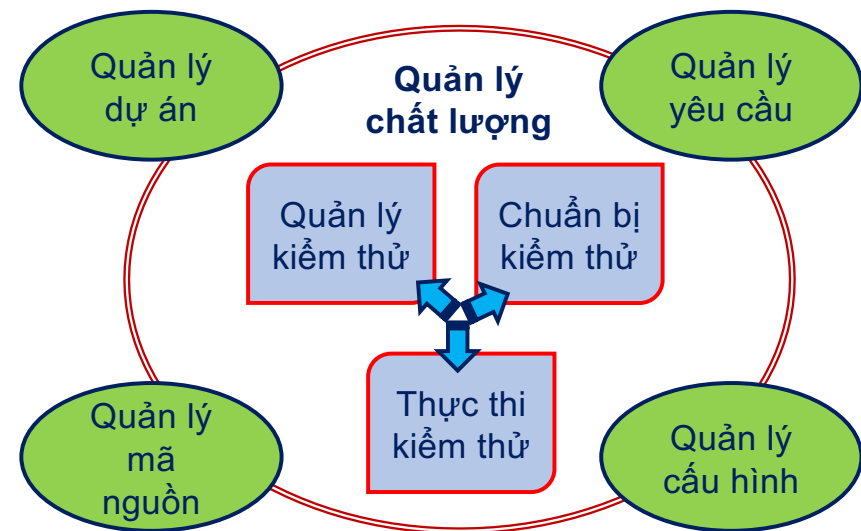
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Kiểm thử

- Kiểm thử có thể được thực hiện trong các pha riêng biệt hoặc các pha lồng nhau



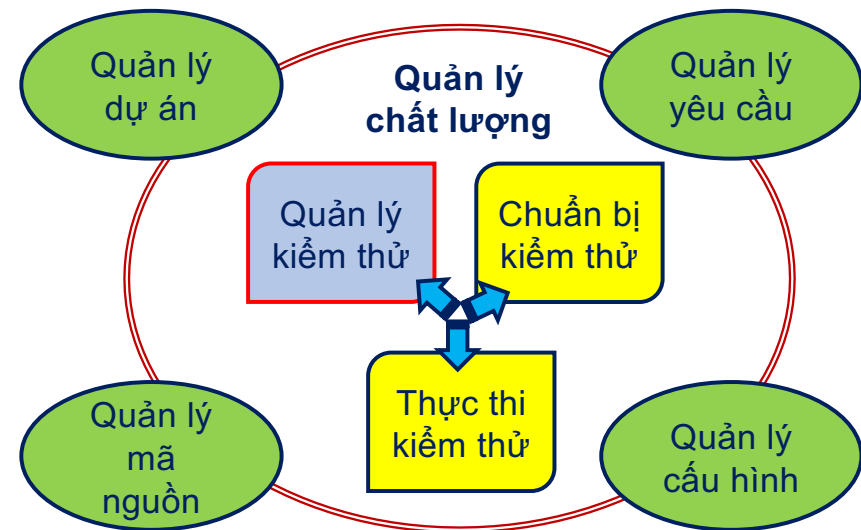
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Quản lý kiểm thử

- Việc quản lý kiểm thử cần được thực hiện ngay từ giai đoạn đầu tiên và kéo dài đến khi dự án kết thúc nhằm đạt được các mục tiêu về chất lượng sản phẩm



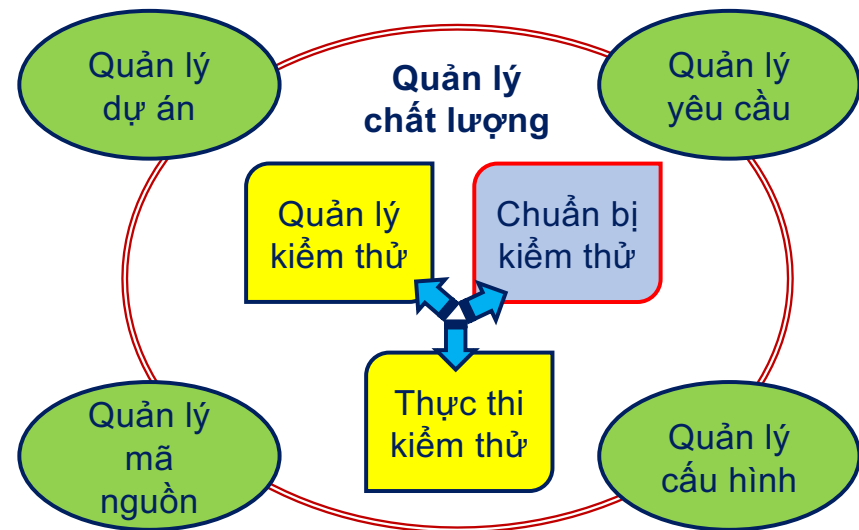
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Chuẩn bị kiểm thử

- Mục tiêu của việc chuẩn bị kiểm thử là để **xác định được các loại kiểm thử** và **môi trường kiểm thử** cần thiết cho từng pha phát triển dự án theo kế hoạch kiểm thử



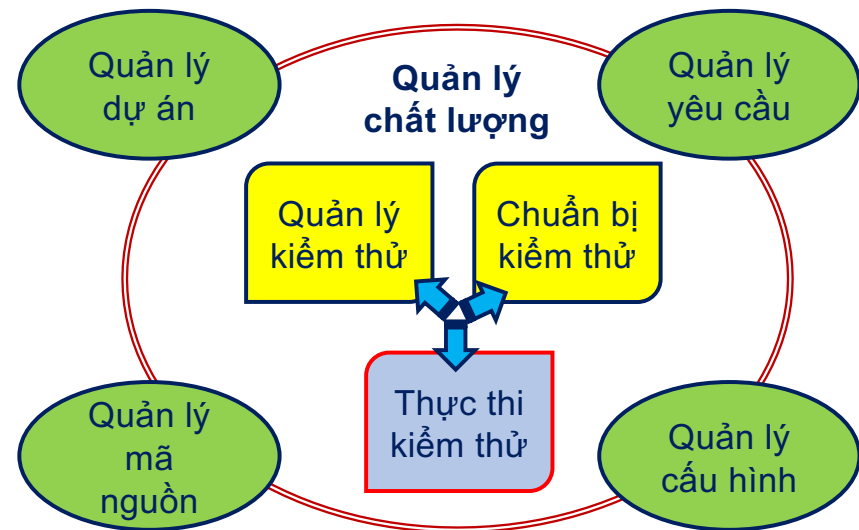
Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

2. QUY TRÌNH ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM



▪ Thực hiện kiểm thử

- Áp dụng trực tiếp các định nghĩa của kiểm thử được đưa ra trong pha chuẩn bị kiểm thử vào quá trình phát triển sản phẩm để tiến hành các hoạt động kiểm thử.



Hình 2.1: Quy trình đảm bảo chất lượng phần mềm

TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học một số **khái niệm cơ bản** trong quy trình đảm bảo chất lượng phần mềm
2. Tiếp sau bài này, **người học có thể tự tìm hiểu thêm** về các khái niệm về bảo trì phần mềm

Bảo trì phần mềm

NỘI DUNG



- 1. Khái niệm bảo trì phần mềm**
- 2. Các hình thức bảo trì**
- 3. Quy trình bảo trì phần mềm**

MỤC TIÊU



Sau bài học này, người học có thể:

1. Hiểu về **khái niệm** bảo trì, các **hình thái** bảo trì phần mềm.
2. Nắm được **quy trình** bảo trì phần mềm.

NỘI DUNG TIẾP THEO



1. Bảo trì phần mềm

- 2. Các hình thái bảo trì phần mềm
- 3. Quy trình bảo trì phần mềm

1. BẢO TRÌ PHẦN MỀM



- Định nghĩa:

- Bảo trì phần mềm là công việc sửa đổi (tu sửa, thay đổi) một phần mềm đã được phát triển (chương trình, dữ liệu, tài liệu), sau khi đã bàn giao để chỉnh các lỗi phát sinh, cải thiện hiệu năng hoặc các thuộc tính, hoặc làm cho phần mềm thích ứng trong một môi trường đã bị thay đổi.
- Các hình thức bảo trì: tu sửa, thích nghi, cải tiến, phòng ngừa.

NỘI DUNG TIẾP THEO



1. Bảo trì phần mềm

2. Các hình thái bảo trì phần mềm

2.1. Bảo trì để tu sửa

2.2. Bảo trì để thích hợp

2.3. Bảo trì để cải tiến

2.4. Bảo trì để phòng ngừa

3. Quy trình bảo trì phần mềm

2. CÁC HÌNH THÁI BẢO TRÌ

2.1. Bảo trì để tu sửa (sửa lỗi)

- Là bảo trì **khắc phục những khiếm khuyết** có trong phần mềm.
- Một số nguyên nhân điển hình:
 - Kỹ sư phần mềm và khách hàng hiểu nhầm nhau.
 - Lỗi tiềm ẩn của phần mềm do sơ ý của lập trình hoặc quá trình kiểm thử chưa được bao quát hết.
 - Vấn đề tính năng: không đáp ứng được yêu cầu về bộ nhớ, tệp, . . .
 - Thiết kế sai, biên tập sai, . . .
 - Thiếu chuẩn hóa trong phát triển phần mềm (trước đó).
- Cần **kiểm tra lại thiết kế để tu sửa** (kỹ nghệ ngược – reverse engineering)

2. CÁC HÌNH THÁI BẢO TRÌ



2.2. Bảo trì để thích hợp

- Là **tu chỉnh phần mềm theo thay đổi của môi trường bên ngoài** nhằm duy trì và quản lý phần mềm theo vòng đời của nó.
- Thay đổi phần mềm **thích nghi với môi trường**: phần cứng, phần mềm.
- Những nguyên nhân chính:
 - Thay đổi về phần cứng (ngoại vi, máy chủ, . . .)
 - Thay đổi về phần mềm (môi trường): thay đổi hệ điều hành (OS)
 - Thay đổi cấu trúc tệp hoặc mở rộng CSDL

2. CÁC HÌNH THÁI BẢO TRÌ



2.3 Bảo trì để cải tiến

- Là việc tu chỉnh phần mềm theo các yêu cầu để ngày càng **hoàn thiện hơn, đầy đủ hơn, hợp lý hơn**.
- Những nguyên nhân chính:
 - Do muốn nâng cao hiệu suất nên cần cải tiến phương thức truy cập.
 - Mở rộng thêm chức năng mới cho hệ thống.
 - Cải tiến nghiệp vụ kéo theo tư liệu vận hành và trình tự công việc.
 - Thay đổi người dùng hoặc thay đổi thao tác.
- Còn **gọi là tái kỹ nghệ** (re-engineering). Mục đích: đưa ra một thiết kế cùng chức năng nhưng có **chất lượng cao hơn**.

2. CÁC HÌNH THÁI BẢO TRÌ



2.4. Bảo trì để phòng ngừa

- Là công việc tu chỉnh chương trình có tính đến việc mở rộng và thay đổi trong tương lai của phần mềm đó.
- Thực ra trong thiết kế phần mềm đã phải tính đến tính mở rộng của nó, nên thực tế ít khi ta bảo trì phòng ngừa nếu như phần mềm được thiết kế tốt.
- Mục đích: sửa đổi để thích hợp với yêu cầu thay đổi sẽ có của người dùng.
- Thực hiện những thay đổi trên thiết kế không tường minh.
- Cần hiểu hoạt động bên trong chương trình và cần thiết kế/lập trình lại.

NỘI DUNG TIẾP THEO



1. Bảo trì phần mềm
2. Các hình thái bảo trì phần mềm
- 3. Quy trình bảo trì phần mềm**

3. QUY TRÌNH BẢO TRÌ



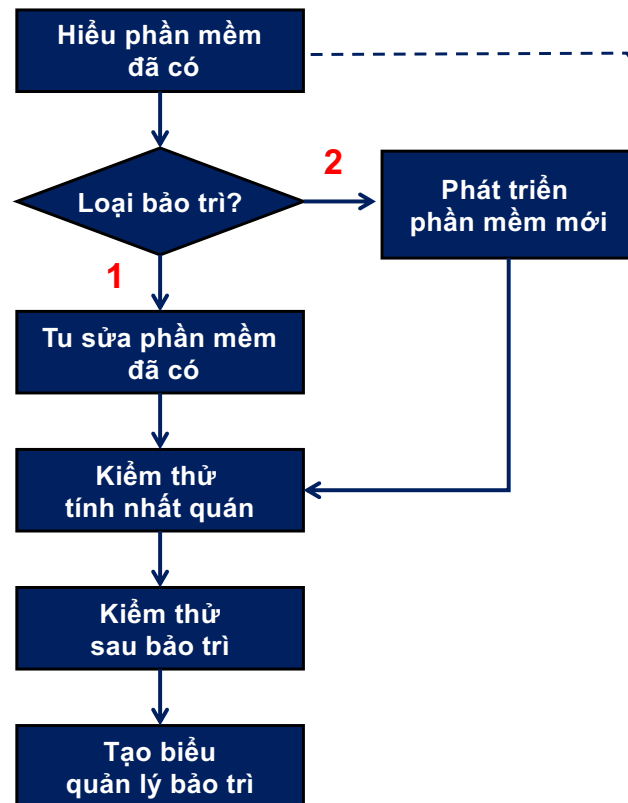
- Cần tuân theo các pha **phân tích, thiết kế, phát triển và kiểm thử** từ khi phát sinh vấn đề cho đến khi giải quyết xong.
- Các nhiệm vụ bảo trì:
 - Phân tích/cô lập: phân tích **tác động**, phân tích những **giá trị lợi ích**, và **cô lập các thành phần cần bảo trì**.
 - Thiết kế: **thiết kế lại hệ thống** (phải biết cách tu sửa, thay đổi).
 - Thực thi: **thay thế mã nguồn** và **kiểm soát** từng đơn vị thành phần hệ thống, có tính đến thời gian lập trình.

3. QUY TRÌNH BẢO TRÌ



- Có 2 loại thao tác bảo trì:
 - Tu chỉnh cái đã có (loại 1)
 - Thêm cái mới (loại 2)

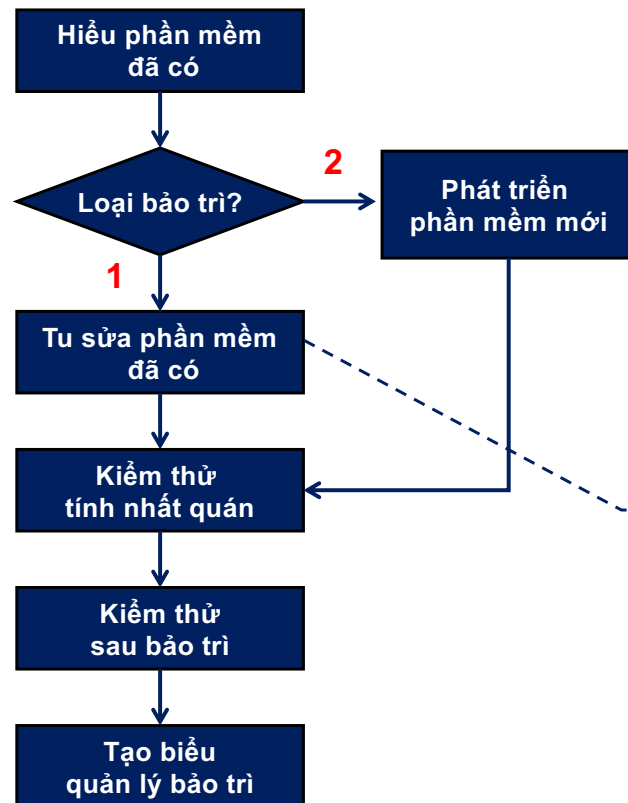
3. QUY TRÌNH BẢO TRÌ



Thực thi:

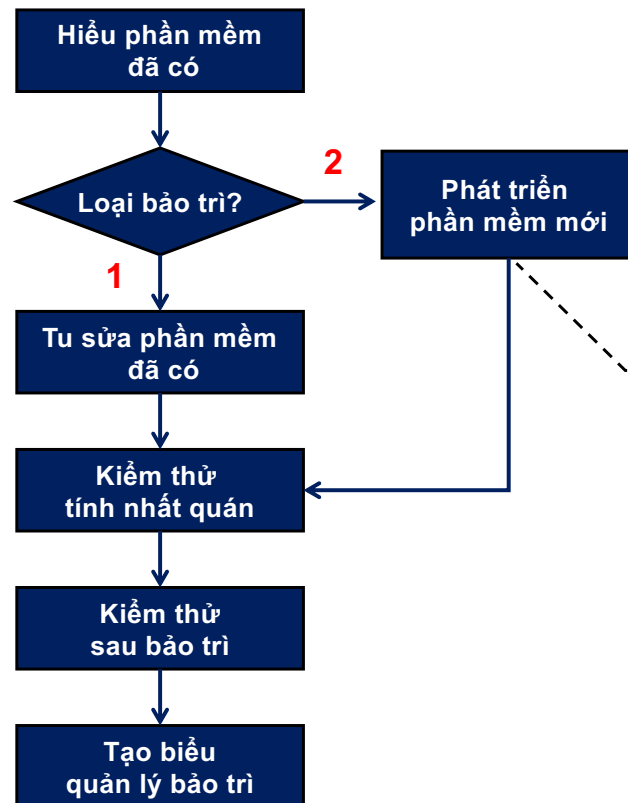
- Nắm vững **các chức năng** của hệ thống theo tài liệu
- Nắm vững **đặc tả chi tiết, điều kiện kiểm thử**,... (theo tài liệu)
- Dò đọc **chương trình nguồn**, hiểu **trình tự xử lý** chi tiết của hệ thống

3. QUY TRÌNH BẢO TRÌ



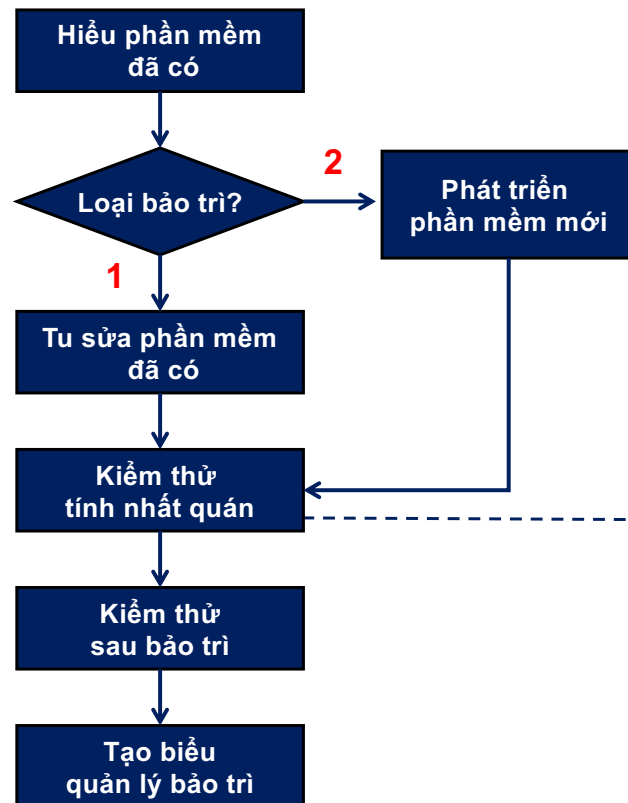
- Tu sửa phần mềm đã có cần **bảo trì chương trình nguồn**, tạo **các module mới** và **dịch lại**.
- Thực hiện **kiểm thử unit** và tu chỉnh những mục liên quan có trong **tài liệu đặc tả**.
- Chú ý theo sát **tác động** của module được sửa đến các thành phần khác trong hệ thống.

3. QUY TRÌNH BẢO TRÌ



- Khi **thêm chức năng mới** phải phát triển chương trình cho phù hợp với yêu cầu
- Cần tiến hành từ **thiết kế, lập trình, gỡ lỗi** và **kiểm thử đơn vị**
- Phản ảnh vào **giao diện** của phần mềm (thông báo, phiên bản,...)

3. QUY TRÌNH BẢO TRÌ

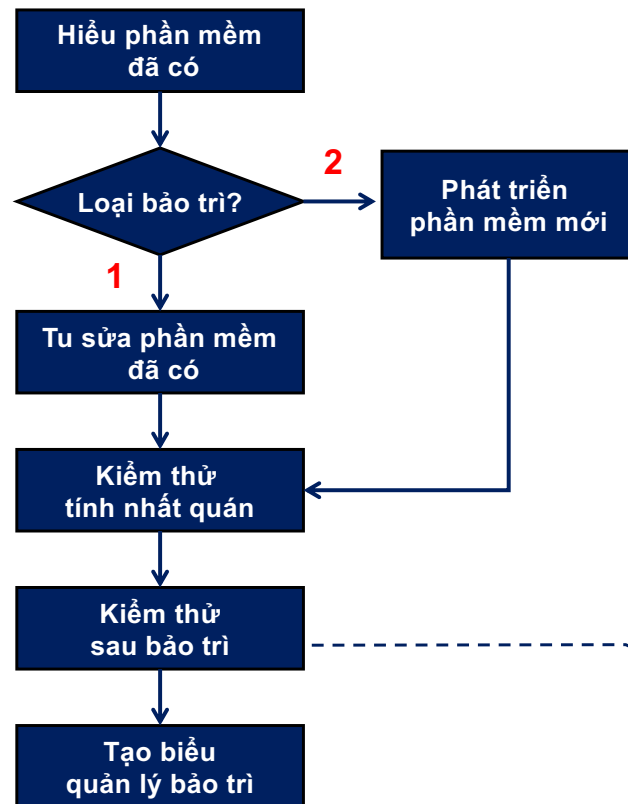


Sử dụng **kiểm thử tích hợp**

- Đưa đơn vị (unit) đã được kiểm thử vào hoạt động trong hệ thống
- Điều chỉnh sự **tương thích** giữa các module
- Dùng các dữ liệu trước đây khi kiểm thử để **kiểm thử lại tính nhất quán**

! Chú ý hiệu ứng làn sóng trong chỉnh sửa

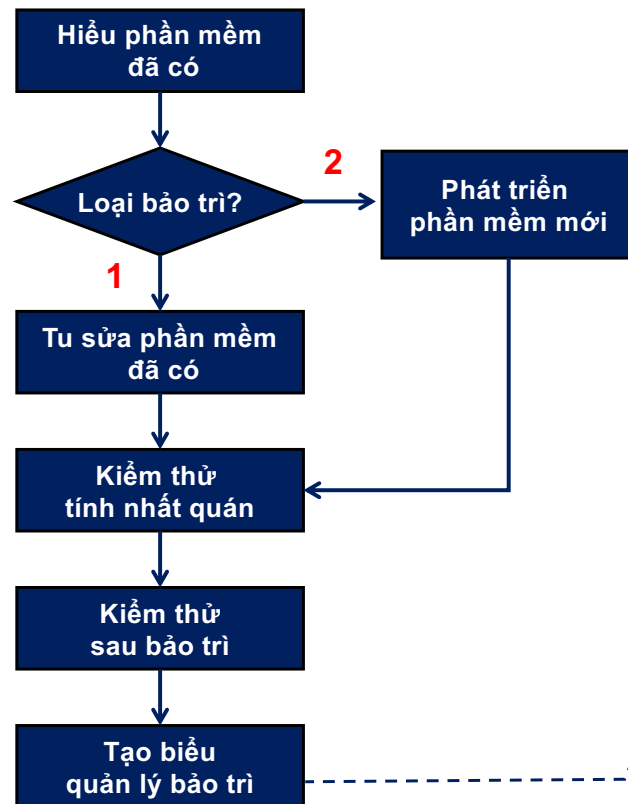
3. QUY TRÌNH BẢO TRÌ



Khi hoàn thành bảo trì:

- Kiểm tra **nội dung mô tả** có trong tài liệu đặc tả
- Xem xét **cách ghi tài liệu** có phù hợp với mô tả môi trường phần mềm mới hay không?

3. QUY TRÌNH BẢO TRÌ



Để quản lý tình trạng bảo trì, **thiết lập các báo biểu:**

- Ngày tháng, giờ
- Nguyên nhân
- Tóm tắt cách khắc phục
- Chi tiết khắc phục, hiệu ứng làn sóng
- Người làm bảo trì, số công, v.v.

TỔNG KẾT VÀ GỢI MỞ



1. Bài học đã cung cấp cho người học một số **khái niệm cơ bản** về bảo trì và **quy trình bảo trì phần mềm**.
2. Người học có thể tự tìm hiểu thêm về sự khác biệt giữa khái niệm kiểm thử và bảo trì phần mềm.



Hướng dẫn bài tập